

帮我深入完整的调查Pencil和OpenDesign的基础代码逻辑, 我要设计一套完全可操控的根据组件库,用AI做设计的东西。

我会先核准 Pencil 和 OpenDesign 对应的具体开源仓库, 再拆解代码结构、数据/组件模型、渲染编辑与导入导出链路, 最后提炼成 AI 可控组件库设计系统方案。OpenDesign 可能有多个同名实现, 我会标出不确定处并优先依据官方仓库和文档判断。

我已把调查分成几条候选主线: 老 Pencil Project / OpenPencil / pencil.dev 需要区分; OpenDesign 基本指向 nexu-io/open-design。接下来会比较“可控画布节点模型”和“Agent 生成 artifact 工作流”哪种更适合你的系统。

已思考 11m 49s >

下面是一版可以直接转成产品技术方案的调研结论。我把你说的 **Pencil** 理解为 **Evolus Pencil Project**, 把 **OpenDesign** 理解为 **nexu-io/open-design / open-design.ai**。OpenDesign 官方说明里也明确把 Open Design、OpenDesign、open-design、opendesign、Open Design AI、OD 视为同一项目, 并指向 `nexu-io/open-design` 仓库。

🔗 Open Design

我的核心判断是: **Pencil** 值得学的是“组件定义系统”, OpenDesign 值得学的是“**AI 编排系统**”。但你真正要做“完全可操控、基于组件库、用 AI 做设计”的东西, 不能让 AI 直接写 HTML/CSS, 也不能只让 AI 在画布上自由摆像素; 你需要中间层: **Design IR / Component Graph**。AI 只允许操作这个受约束的组件图, 最终渲染、导出、校验都由你自己的引擎完成。

1. Pencil 的基础代码逻辑: 本质是「Stencil/ShapeDef → Document/Page → Exporter」

Pencil V3 是一个 Electron 桌面应用, 官方 README 说明 V3 从 Mozilla XULRunner 重写到 Electron, 并加入了新的 zip 格式、页面管理、树状页面模型、嵌入字体、PDF/打印/导出等能力。🔗 GitHub 它的 `package.json` 也显示依赖 Electron, 应用入口脚本通过 Electron 启动, 并关联 `.ep`、`.epz`、`.epgz` 等文件格式。🔗 GitHub 🔗 GitHub

从目录上看, Pencil 的代码可以拆成三层: 外层是 Electron app 和 UI, 核心逻辑在 `app/pencil-core`, 组件库在 `app/stencils`, 导出逻辑在 `exporter` 目录。仓库目录里能看到 `archive`、`lib`、`pencil-core`、`stencils`、`theme`、`views` 等应用结构; `pencil-core` 下又分为 `definition`、`document`、`editor`、`exporter`、`propertyType`

等模块；`stencils` 下有 Android、iOS、Web、Flowchart、SketchyGUI 等组件集合。

 [GitHub](#)

 [GitHub](#)

 [GitHub](#)

Pencil 最核心的设计思想是：组件不是一段最终 UI，而是一个 **Shape Definition**。

`ShapeDef` 里包含 `id`、`displayName`、`contentNode`、`propertyGroups`、`behaviors`、`actions`，并维护属性、行为、动作的 map。 [GitHub](#) 属性系统里有 `PropertyGroup`、`Property`，行为系统里有 `Behavior`、`BehaviorItem`、`BehaviorItemArg`；其中 `BehaviorItemArg` 会把 `$property` 这类引用预处理成 `properties.xxx`，并把属性依赖关系记录到 `relatedTargets` 等结构里。 [GitHub](#)

可以把 Pencil 的内部模型理解成这样：

```
Stencil Collection
└─ ShapeDef
    │ contentNode          # 组件的 SVG/XHTML/DOM 内容结构
    │ propertyGroups      # 可编辑属性分组
    │   └─ Property       # 类型、初始值、依赖关系、meta
    │ behaviors          # 属性变化如何影响视觉节点
    │   └─ BehaviorItem  # target + handler + args
    └─ actions            # 菜单动作 / 快捷行为

Document
└─ Pages
    └─ Page XML
        │ Properties
        └─ Content        # canvas drawingLayer 中的节点

Exporter
└─ SVG
└─ Rasterized
└─ Web Page
└─ Printing
└─ ODT / Office conversion
```



Pencil 的文档层也是 XML/DOM 风格。`PencilDocument.prototype.toDom()` 会创建 `<Document>`，里面包含 `<Properties>` 和 `<Pages>`，每个页面通过 `href` 指向页面文件；`Page` 本身有 `id`、`name`、`width`、`height`、背景页、缩放、滚动位置等属性；`Page.prototype.toXml()` 会生成 `<p:Page>`，再把画布 `drawingLayer` 里的节点导入到页面内容里。 [GitHub](#)  [GitHub](#) 导出层则有 `svgExporter`、`webPageExporter`、`printingExporter`、`baseRasterizedExporter`、`odtExporter` 等。 [GitHub](#)

Pencil 对你最有价值的部分

你要重点借鉴 Pencil 的这几个点：

第一，组件库是声明式的。Pencil 的 stencil/shape 不是用户随便画出来的一堆 DOM，而是一个可解析、可注册、可带属性和行为的定义。这非常适合你要做的 AI 组件库。

第二，属性是组件的主要控制面。AI 不应该说“把这个东西变好看”，然后直接改 CSS；它应该说“把 `Hero.variant` 从 `split` 改成 `centered`，把 `density` 改成 `spacious`，把 `title.maxLines` 控制在 2 行”。

第三，行为表达了属性和视觉节点之间的关系。Pencil 的 `BehaviorItemArg` 和 `$property` 依赖机制很值得迁移成现代版。你的系统里也需要知道：当 `button.size = lg` 时，padding、font-size、height、icon gap 会如何联动。

第四，导出是独立管线。Pencil 不是只有一个显示层，它有导出器。你的系统也应该从一开始就把“设计源数据”和“导出目标”分开：React、HTML、SVG、PDF、PPTX、图片都只是不同 exporter。

Pencil 不建议照搬的部分

不要直接复制 Pencil 的 XML/DOM 风格作为现代 AI 设计系统的核心。它适合传统桌面原型工具，但对 AI 原生系统来说会有几个问题：类型约束弱、运行时 DOM 容易变成事实数据源、组件版本管理不清晰、AI patch 难以保证安全和可预测。你应该吸收它的 **组件定义思想**，但把底层升级成 **TypeScript + JSON Schema/Zod + versioned component manifest + deterministic renderer**。

2. OpenDesign 的基础代码逻辑：本质是「Skill + DESIGN.md + Agent Adapter + Artifact Pipeline」

OpenDesign 是一个本地优先的 AI 设计工作区。README 把它描述成开源 Claude Design 替代品，提供 macOS/Windows 桌面应用、100+ skills、150 个 DESIGN.md systems、插件和导出能力；它能生成 web/desktop/mobile prototypes、dashboards、decks、images/video/HyperFrames，并通过沙盒 iframe 预览，支持 HTML、PDF、PPTX、MP4 导出，也支持多种本地 CLI agent 和 OpenAI-compatible endpoint。  [GitHub](#)

OpenDesign 的核心不是一个传统画布编辑器，而是一个 **AI agent orchestration shell**。官方架构文档把它拆成三种部署拓扑：默认完全本地，浏览器连接 Next dev server，再连接本地 `od` daemon，由 daemon 启动 agent CLI；也可以 web 在 Vercel、本地 daemon 在用户机器；还可以 web 直接走 API 但体验会降级。  [GitHub](#) 架构图里核心模块包括 web app、transport、daemon、session manager、skill registry、agent adapter pool、design-system resolver、artifact store、preview compile pipeline、export pipeline 等。  [GitHub](#)

OpenDesign 的 daemon 是关键：它监听 `localhost:7456`，提供 REST/SSE，管理 session、active agent、skill、artifact、tool calls、design system；它会扫描 skills、写

artifact 到磁盘、做 preview compile 和 export hooks。  GitHub Agent adapter 负责检测和启动用户已有的 CLI agent，把 SKILL.md、DESIGN.md、用户 prompt 和工作目录交给 agent，然后把 stdout/stderr 流转换成结构化事件。  GitHub

OpenDesign 的一次生成流程大致是：

```
User Prompt
→ 选择 mode / skill
→ daemon 加载 SKILL.md
→ daemon 加载 DESIGN.md
→ 创建 .od/artifacts/<slug>/
→ agent adapter 启动 Claude Code / Codex / Gemini CLI 等
→ agent 在 artifact 目录写文件
→ daemon 通过 SSE 推送 tool/text/file events
→ web app 更新 artifact tree 和 iframe preview
→ 用户通过 chat/comment/slider refine
→ export HTML/PDF/PPTX/ZIP/Markdown
```

这个流程和官方 data flow 基本一致：web 发起 session.generate，daemon 选择 skill、加载 DESIGN.md、创建 artifact 目录，用 SKILL.md + DESIGN.md + user prompt + cwd 调 agent；agent 产出的流式事件回到 web；完成后写 history；用户评论或滑杆调整会触发 refine。  GitHub 预览层使用 sandboxed iframe，HTML 通过 srcdoc 渲染，JSX 通过 vendored React 18 + Babel 编译，agent 写文件会触发 debounced rebuild。  GitHub

OpenDesign 的 Skill 协议也很重要。它采用 Claude Code 的 SKILL.md 约定，最小结构是一个 skill 目录，里面有 SKILL.md，也可以带 assets、references、脚本等。frontmatter 里可以有 name、description、trigger 等；OpenDesign 又扩展了 mode、preview、输入字段、live tweak parameters、outputs、capabilities_required 等设计相关 metadata。

 GitHub 官方 spec 也把它的核心产品定义为：通过编排用户已有的 code agent，把自然语言 brief 变成可编辑、可预览的设计 artifact；它押注于现有 code agent CLI、文件化 SKILL.md、DESIGN.md 设计系统，以及通过丢文件夹扩展 skill。  GitHub

OpenDesign 的 mode 分为 Prototype、Deck、Template、Design System。Prototype mode 会选择 skill、输入 prompt、流式显示 tool-call feed、artifact tree、iframe preview、comment mode、sliders，并支持导出 HTML/PDF/ZIP；Design System mode 则先生成或维护 DESIGN.md，其他模式再读取它。  GitHub

OpenDesign 对你最有价值的部分

你要重点借鉴 OpenDesign 的这几个点：

第一，AI 能力应该插件化。 不要把“生成落地页”“生成 dashboard”“生成 PPT”“提取设计系统”全写进一个大 prompt。应该像 OpenDesign 一样，用 skill 把不同设计任务拆成独立能力包。

第二，设计系统应该文件化。OpenDesign 用 `DESIGN.md` 做设计系统注入，这个方向是对的。你的系统可以更进一步：`DESIGN.md` 给人读，`tokens.json` / `components.manifest.json` / `rules.json` 给机器读。

第三，**agent adapter** 很关键。你不一定要绑定某一家模型。OpenDesign 的 adapter 思想是：检测不同 agent 的能力，统一成 `thinking`、`tool_call`、`tool_result`、`text_delta`、`file_write`、`error`、`done` 这类事件。文档里的 adapter interface 也明确包含 `detect`、`capabilities`、`run`、`cancel`、`resume` 和 `AgentEvent` 类型。  [GitHub](#)

第四，**artifact** 要落盘、可回放、可 diff。OpenDesign 的 artifact store 使用 `.od/artifacts/<slug>/artifact.json/index.html/assets`、`history.jsonl`、`sessions` 等文件结构，理由是 plain files、git-friendly、无 DB 依赖。  [GitHub](#) 这对你非常重要，因为 AI 设计必须能回滚、对比、审计。

OpenDesign 不足以满足“完全可控”的地方

OpenDesign 更像一个 AI 设计生成/编排工作台，不是一个强约束的组件图编辑内核。它非常擅长把 `prompt`、`skill`、`DESIGN.md`、`agent`、`preview`、`export` 组织起来；但如果你要的是“组件级完全可控”，还需要自己补一个核心层：组件图是唯一真相。

也就是说，OpenDesign 的默认路径更接近：

```
AI → 写 HTML / JSX / slides.json / assets → preview / export
```



你要做的应该是：

```
AI → 受限操作 Component Graph / Design IR
    → schema validator
    → layout solver
    → renderer
    → preview / export
```



这个区别非常大。前者更自由，但难控；后者更像真正的 AI-native Figma/Webflow/Framer 内核。

3. 你应该设计的系统：组件库不是素材库，而是“可操作协议”

我理解你说的“根据组件库”是“基于组件库”。这个系统的核心原则应该是：

AI 不生成最终设计文件；AI 生成对组件图的操作。

组件库不是 UI 资产仓库；组件库是 AI 可以调用的设计 API。

建议你把系统拆成 9 个核心模块。

4. 推荐架构：AI Design Kernel

4.1 Component Registry：组件注册中心

每个组件都要有机器可读的 manifest。它不只是 React 组件，也不只是 Storybook demo，而是包含：

```
component id
version
props schema
variants
slots
states
layout constraints
token usage
allowed children
forbidden usage
renderers
examples
AI affordances
QA rules
migration rules
```



示例：

</> YAML



```
id: ui.hero
version: 1.2.0
displayName: Hero Section

props:
  title:
    type: string
    required: true
    maxLength: 80
  subtitle:
    type: string
    maxLength: 180
  media:
    type: asset
    allowed: [image, video]
  density:
    type: enum
    values: [compact, regular, spacious]

variants:
  - split
  - centered
```

```

- editorial

slots:
  actions:
    accepts: [ui.button, ui.button-group]
    max: 2
  media:
    accepts: [ui.image, ui.video, ui.illustration]
    max: 1

tokens:
  consumes:
    - color.bg
    - color.text
    - space.*
    - radius.*
    - typography.*

constraints:
  - title.maxLines <= 3
  - actions.count <= 2
  - media.required when variant == "split"

renderers:
  react: ./renderers/react/Hero.tsx
  html: ./renderers/html/Hero.ts
  figmaLike: ./renderers/figma/Hero.ts

ai:
  intents:
    - landing_above_fold
    - product_intro
    - campaign_header
  examples:
    - ./examples/hero-saas.json
    - ./examples/hero-editorial.json
  forbidden:
    - inline-style
    - arbitrary-absolute-position
    - unregistered-color

```

这就是 Pencil `ShapeDef + Property + Behavior` 思想的现代化版本，但比 Pencil 更类型化、更版本化、更适合 AI。

4.2 Design Tokens / Brand System：品牌与设计变量

建议同时保留两套表达：

```
DESIGN.md      # 给人和 LLM 读
tokens.json    # 给渲染器和 validator 读
rules.json     # 给 QA / policy / lint 读
```



`DESIGN.md` 可以写品牌语气、视觉原则、竞品参考、禁用风格；`tokens.json` 管颜色、字号、间距、圆角、阴影、断点；`rules.json` 管比如“主按钮每屏最多 1 个”“金融产品不能用过度游戏化视觉”等规则。

OpenDesign 的 `DESIGN.md` 注入方向值得借鉴，但你要进一步把它结构化，否则 AI 能读，人也能读，但系统本身无法严格校验。

4.3 Design IR / Component Graph：唯一真相层

这是你系统的核心。所有设计都应该存成一个组件图，而不是直接存 HTML、SVG 或 canvas DOM。

示例：

<> JSON



```
{
  "schemaVersion": "0.1",
  "project": {
    "name": "AI CRM Landing Page",
    "theme": "brand.default"
  },
  "artboards": [
    {
      "id": "home.desktop",
      "width": 1440,
      "height": "auto",
      "children": [
        {
          "id": "hero1",
          "type": "ui.hero",
          "variant": "split",
          "props": {
            "title": "AI-native CRM for founders",
            "subtitle": "Turn every customer conversation into a sales workflow.",
            "density": "spacious"
          },
          "slots": {
            "actions": [
              {
                "id": "cta1",
                "type": "ui.button",
                "variant": "primary",
```

```

        "props": {
          "label": "Get started"
        }
      },
      {
        "id": "cta2",
        "type": "ui.button",
        "variant": "secondary",
        "props": {
          "label": "View demo"
        }
      }
    ]
  },
  "layout": {
    "mode": "section",
    "maxWidth": 1200,
    "paddingY": "space.9"
  }
}
]
}
]
}

```

这层要支持：

```

node id
component type
variant
props
slots
children
layout
responsive overrides
states
interactions
data bindings
asset references
provenance
lock flags
comments
history

```



重点：AI 只能改这个 IR，不能绕过它直接改最终 HTML。

4.4 Operation / Patch Protocol: AI 的唯一操作接口

不要让 AI 直接返回完整 JSON 文件，更不要让它直接写任意代码。让它返回领域操作：

</> JSON



```
[
  {
    "op": "setProp",
    "nodeId": "hero1",
    "path": "props.title",
    "value": "Design with components, not pixels"
  },
  {
    "op": "setVariant",
    "nodeId": "hero1",
    "variant": "editorial"
  },
  {
    "op": "insertNode",
    "parentId": "home.desktop",
    "after": "hero1",
    "node": {
      "id": "features1",
      "type": "ui.feature-grid",
      "variant": "3-col",
      "props": {
        "title": "Everything your team needs"
      }
    }
  }
]
```

建议支持这些操作：

```
insertNode
deleteNode
moveNode
wrapNode
unwrapNode
setProp
setVariant
setSlot
bindData
setToken
setResponsiveOverride
setInteraction
setState
lockNode
unlockNode
```



每个操作都必须经过：

```
schema validation
component compatibility check
slot constraints check
token compliance check
layout validation
accessibility check
diff preview
human approval, 可选
```



这样你的 AI 才是“可控操作员”，不是“自由画图机器”。

4.5 Layout Engine：确定性布局引擎

Pencil 更偏画布/节点；OpenDesign 更偏 artifact 文件生成。你的系统需要一个强约束布局层。

建议初期不要做完全自由画布，而是先支持这几种 layout primitive：

```
Stack
Grid
Section
Container
Columns
AutoLayout
AbsoluteLayer, 只用于 hero 装饰/海报类场景
```



每个组件声明自己的布局行为：

YAML



```
layout:
  display: section
  childrenFlow: vertical
  responsive:
    desktop: split
    tablet: stack
    mobile: stack
  minWidth: 320
  maxWidth: 1440
```

你的布局引擎要保证：

```
不会重叠
不会溢出
移动端可读
文本长度超限时策略
```



组件 slot 不被错误填充
视觉 tokens 合法

初期可以用 CSS Flex/Grid 做确定性布局；如果后面要做更复杂的画布约束，再考虑 Yoga、Taffy、Cassowary/Kiwi 这类约束求解思路。

4.6 Renderer / Exporter：多目标输出

最终输出不要只有一个 React 页面。建议至少设计这些 renderer：

```
render-react  
render-html  
render-svg  
render-canvas-preview  
render-pdf  
render-pptx  
render-image
```

核心原则：

```
Design IR 是源数据  
Renderer 是编译目标  
Exporter 是交付目标
```

例如：

```
Component Graph  
→ React renderer      → Next/Vite project  
→ HTML renderer       → static HTML  
→ SVG renderer        → poster/card  
→ PPTX exporter       → sales deck  
→ PDF exporter        → proposal/report  
→ Image exporter      → marketing asset
```

这点可以借鉴 Pencil 的 exporter 分层，也可以借鉴 OpenDesign 的 artifact/export pipeline。

4.7 Preview + Inspector：每个视觉节点都能回到组件图

渲染时每个节点都要带追踪信息：

HTML

</>

▶

📄

```
<section
  data-node-id="hero1"
  data-component="ui.hero"
  data-variant="split"
>
```

这样用户在预览里点一个元素时，系统可以知道：

```
这是哪个 component node
属于哪个 slot
来自哪个 prop
由哪个 token 控制
是否被锁定
AI 能改哪些字段
```



OpenDesign 已经有 comment mode 和 `[data-od-id]` 的思路，你可以把它升级成完整 inspector。

4.8 AI Orchestrator：规划、检索、补丁、校验、修复

AI 编排层不要只是一条 prompt。建议拆成：

```
Intent Parser
Component Retriever
Patch Planner
Patch Generator
Validator
Visual QA
Repair Loop
Explainer
```



一次请求应该这样跑：

用户：帮我做一个高端 SaaS 首页



1. Intent Parser
 - 类型：landing page
 - 风格：高端、SaaS、B2B
 - 目标：转化
 - 输出：web prototype
2. Component Retriever
 - 候选：hero, logo-cloud, feature-grid, testimonial, pricing, faq, footer
3. Patch Planner
 - 生成页面结构计划

4. Patch Generator
 - 只输出 component graph operations
5. Validator
 - 检查组件 schema、slot、tokens、布局、ally
6. Renderer
 - 生成 preview
7. Visual QA
 - 检查是否溢出、是否有空态、CTA 是否缺失
8. Repair Loop
 - 自动修正不合法 patch
9. User Diff
 - 用户确认或继续自然语言修改

4.9 Artifact Store / History: 可回滚、可审计、可复现

建议项目目录长这样:

```
my-design-project/  
  DESIGN.md  
  tokens.json  
  components.lock.json  
  design.graph.json  
  rules.json  
  
  .aid/  
    artifacts/  
      home/  
        index.html  
        preview.png  
        artifact.json  
      history.jsonl  
      sessions/  
      snapshots/  
      qa-reports/  
  
  exports/  
    home.html  
    home.pdf  
    pitch-deck.pptx
```



history.jsonl 每一行记录一次 AI 或人工操作:

</> JSON



```
{
  "time": "2026-06-11T10:30:00+09:00",
  "actor": "ai",
  "intent": "make hero more premium",
  "ops": [
    {"op": "setVariant", "nodeId": "hero1", "variant": "editorial"}
  ],
  "beforeHash": "abc",
  "afterHash": "def",
  "validation": "passed"
}
```

这样你可以做到：

undo / redo

版本对比

设计决策追踪

AI 输出审计

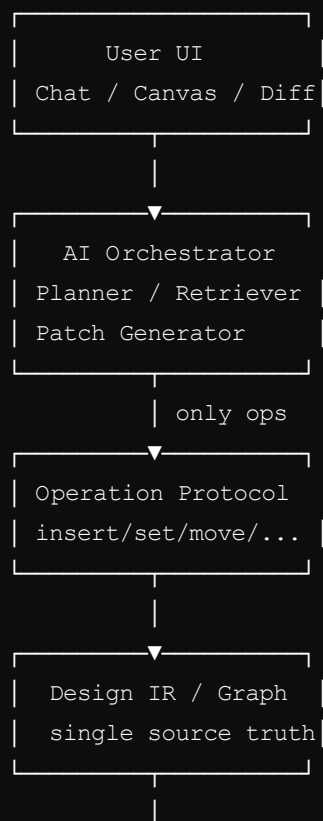
团队协作

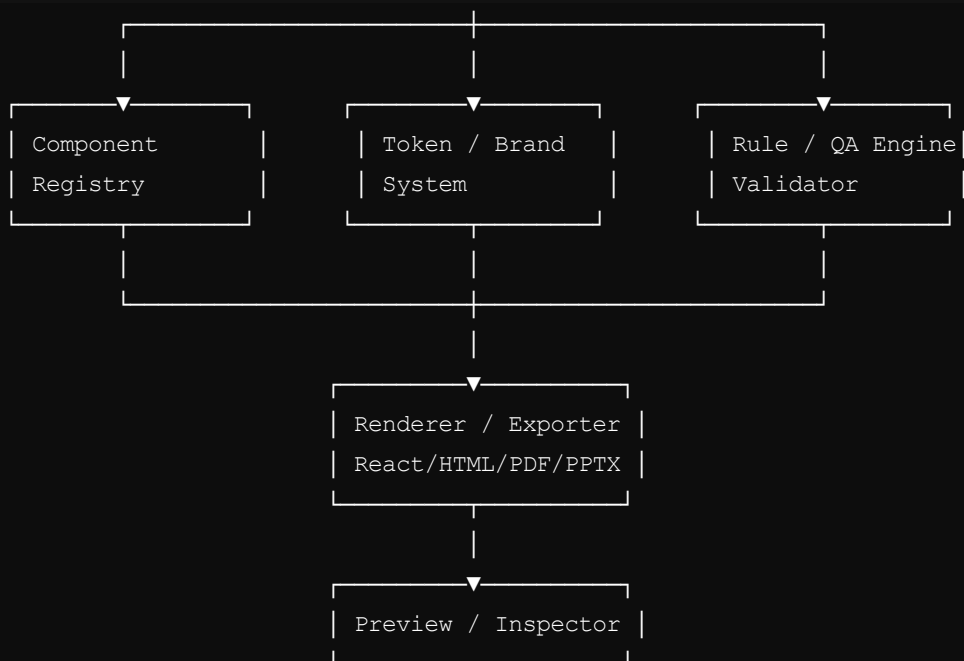
自动测试



5. Pencil + OpenDesign 合体后，你真正需要的产品内核

可以把目标架构画成这样：





一句话：

Pencil 给你“组件可控性”的启发，**OpenDesign** 给你“AI 工作流”的启发，你要补的是“AI 可操作组件图协议”。

6. 最小可行产品建议

不要一开始做完整 Figma。建议 MVP 只做 网页/落地页/产品原型，因为这个场景最适合组件化和 AI 生成。

Phase 0：组件协议

先定义：

```
component.schema.ts
design-graph.schema.ts
operation.schema.ts
tokens.schema.ts
```



技术建议：

```
TypeScript
Zod 或 JSON Schema
pnpm workspace
Vitest
```



先做 10 个核心组件：

```
ui.page
ui.section
```



```
ui.hero
ui.navbar
ui.button
ui.card
ui.feature-grid
ui.testimonial
ui.pricing
ui.footer
```

Phase 1: 可视化预览 + Inspector

实现:

```
Design Graph → React Preview
点击节点 → 右侧属性面板
属性修改 → 生成 operation
operation → validator → graph update → rerender
```

这一步完成后，你已经有了一个“小型受控设计编辑器”。

Phase 2: 接入 AI，但只能输出 patch

不要让模型写页面代码。系统 prompt 里明确:

```
You are not allowed to output HTML/CSS.
You can only call design operations.
Every operation must satisfy the component registry.
```

AI 生成后先进入 diff:

```
新增了哪些 section
改了哪些 props
用了哪些组件
违反了哪些规则
```

用户确认后再应用。

Phase 3: 设计系统生成

做一个类似 OpenDesign Design System mode 的能力:

```
输入: 品牌说明 / 官网 / 截图 / Figma 导出 / PDF brand guide
输出:
  DESIGN.md
  tokens.json
  component preferences
  forbidden patterns
  example compositions
```

但注意：`DESIGN.md` 只是自然语言层，真正控制系统的是 `tokens.json` 和 `rules.json`。

Phase 4: 导出

优先级建议：

1. static HTML
2. React package
3. PDF
4. PNG screenshot
5. PPTX
6. Figma-like JSON, 后续再做



7. 技术栈建议

比较稳的工程结构：

```
apps/  
  web/           # 设计 UI、chat、preview、inspector  
  daemon/        # 本地服务、agent adapter、artifact store  
  
packages/  
  ir/            # Design Graph schema, operations  
  registry/      # component registry loader  
  tokens/        # token parser / transformer  
  render-react/  # React renderer  
  render-html/   # HTML renderer  
  validator/     # schema/layout/ally/rule validation  
  agent/         # LLM/agent adapters  
  exporter/      # pdf/pptx/png/html exports  
  examples/      # fixture projects
```



推荐技术：

```
TypeScript  
Zod / JSON Schema  
React  
Vite 或 Next.js  
Hono / Fastify  
Playwright 或 Puppeteer  
axe-core  
pnpm workspace  
SQLite, 可选, 仅做索引和缓存  
plain files, 作为项目主存储
```



组件库可以用 React 实现，但组件 manifest 必须框架无关。否则你会被 React 绑定住，后面要导出 HTML、PPTX、SVG、Figma-like JSON 会很痛苦。

8. 最关键的产品控制能力

你要从第一天就支持这些“控制旋钮”：

作用范围：

- 整页
- 当前 section
- 当前组件
- 当前 prop
- 当前 token

自由度：

- strict：只能使用现有组件和 tokens
- guided：可以组合现有组件，但不能发明新组件
- creative：可以提出新组件草案，但必须进入人工审核

锁定：

- lock content
- lock layout
- lock brand
- lock component
- lock selected nodes

变更方式：

- diff first
- apply immediately
- generate alternatives
- explain changes

这会让用户感觉 AI 是“可控设计搭档”，不是“随机生成器”。

9. 最重要的设计原则

你这个产品最容易失败的点，是把 AI 设计理解成“生成漂亮页面”。真正的壁垒应该是：

把设计变成一套 AI 可理解、可验证、可回滚、可组合、可导出的操作系统。

所以底层不要围绕 prompt 设计，而要围绕这几个东西设计：

```
Component Registry
Design Tokens
Design Graph
Operation Protocol
Validator
Renderer
Artifact History
```

Pencil 的启发是：组件必须有属性、行为、定义和导出。

OpenDesign 的启发是：AI workflows 必须 skill 化、本地化、artifact 化、agent-adapter 化。

你的核心创新应该是：AI 不能越过设计库，只能通过操控操作。

pencil.dev 的。因为我是做 AI 项目，所以肯定是参考

最后补一句边界说明：这版 pencil.dev 而不是你调查的这个。

本地运行后的断点级动态调试报告。进入实现前，下一步最值得做的是把上面的

你说得对，我前面把 Pencil 误判成 Evolus Pencil，这是方向性错误。现在应按 pencil.dev component manifest / design graph / operation protocol 三个 schema 先定下来，重新核对，并与 OpenDesign 一起放进“AI 可控组件库设计系统”的框架里分析。因为它们会决定整个产品的大天花板。

我已核对到：pencil.dev 不是开源源码级项目，而是 .pen JSON 设计文件、MCP 工具层、CLI/headless、IDE/桌面同一编辑引擎。后面会按公开协议和行为反推其基础逻辑。

已思考 9m 0s >

你这个纠正是对的，而且影响很大。上一版里我把 Pencil 查成了老的 Evolus Pencil，这对你要做的 AI 设计产品基本没有参考价值。pencil.dev 才是应该重点研究的对象：它不是“传统画图软件”，而是 IDE/桌面/CLI 三端共用的 AI 可操作设计画布，核心是 .pen JSON 文件格式、本地 MCP 读写工具、组件/实例/槽位、变量/主题、代码同步和 headless CLI。官方文档也明确写了：Pencil 是一个集成进开发环境的 vector design tool，用来让设计、组件、代码库和 AI assistant 保持一致。

下面我把之前错误的 Pencil 部分彻底替换成 pencil.dev 的正确基础代码逻辑调查，再给你一套更贴近你目标的产品内核设计。

1. 先定性：pencil.dev 的真正定位

pencil.dev 的核心不是“AI 生成图片”，也不是“AI 生成 HTML”。它更像是：

```
开放 .pen 设计文件
+ 无限画布编辑器
+ 本地 MCP Server
+ AI 可读写工具集
+ 设计变量 / 组件 / slot / ref 机制
+ design ↔ code 双向同步
+ headless CLI 自动化
```

官网的产品表述非常直接：Pencil 是一个 agent-driven MCP canvas，设计文件放在代码库里，支持 Cursor、VS Code、Claude Code、OpenAI Codex 等工具；它强调“open design format lives in your codebase”、“bi-directional MCP vector canvas”，并且不只是提供 MCP reading tools，还提供 full write access 来操作画布。

所以它和 OpenDesign 的区别是：

OpenDesign: 偏 AI artifact 工作台
Pencil.dev: 偏 AI 可操作的设计文件 + 画布内核



你要做“基于组件库、完全可操控、用 AI 做设计”的产品，**Pencil.dev** 的参考价值比 **OpenDesign** 更底层。OpenDesign 更值得参考 agent workflow、skill、artifact；Pencil.dev 更值得参考 **AI 如何通过工具协议控制设计画布**。

2. 重要边界：Pencil.dev 源码不是公开仓库

这里要先讲清楚：pencil.dev 的核心源码目前不能像 OpenDesign 一样直接审仓库。Pencil 官方 AI Integration 文档写了 “Repository is private: Source code not yet public”。

所以对 Pencil.dev 的“基础代码逻辑”不能声称是源码级反编译结论。更准确地说，能可靠调查的是：

1. 官方产品架构暴露出来的运行方式
2. .pen 文件格式和 TypeScript schema
3. MCP 工具集
4. CLI 工作方式
5. AI / IDE / 桌面 / headless 的交互模型
6. 设计组件、变量、slot、script node 的数据模型



这已经足够反推出它的核心内核形态。

3. Pencil.dev 的基础代码逻辑：.pen 是唯一核心数据结构

Pencil 的 .pen 文件是 JSON-based、version-control friendly、portable 的设计文件格式，官方建议把 .pen 放在项目 workspace 里，像代码一样用 Git 管理。

开发者文档进一步说明：.pen 文件包含一个 JSON 结构，描述的是一个 object tree，类似 HTML 或 SVG；每个对象都是无限二维画布上的图形实体；每个对象需要有唯一 id 和 type，比如 rectangle、frame、text、script 等。

也就是说，Pencil.dev 的核心不是这样的：

Prompt → HTML/CSS



而是这样的：

Prompt / Agent / Human edit
→ MCP operation
→ .pen JSON object tree



→ Editor engine render
→ Screenshot / export / code sync

你可以把 `.pen` 理解成它的 **Design IR**:

```
</> JSON   
  
{  
  "version": "2.13",  
  "themes": {},  
  "imports": {},  
  "variables": {},  
  "children": [  
    {  
      "id": "hero",  
      "type": "frame",  
      "x": 0,  
      "y": 0,  
      "width": 1440,  
      "height": 900,  
      "layout": "vertical",  
      "children": []  
    }  
  ]  
}
```

这对你特别关键：**AI 不应该直接改最终代码，而应该改一个可验证、可渲染、可版本化的设计中间层。**

4. `.pen` 的对象模型：更像 Figma Node Tree + HTML/SVG + Flexbox

Pencil 的 object tree 里，顶层对象放在无限二维画布上，用 `x`、`y` 表示左上角位置；嵌套对象相对父级定位；父级可以用类似 flexbox 的 layout system 接管子元素尺寸和位置，例如 `layout`、`justifyContent`、`alignItems`；子元素可以固定尺寸，也可以填充父容器或 fit content。  Pencil

这说明 Pencil 的基础布局逻辑是：

```
Canvas absolute positioning  
+ Nested relative positioning  
+ Frame flex layout  
+ Fill / fixed / fit_content / fill_container sizing 
```

它的 schema 里 `Layout` 明确支持：

❏ TypeScript



```
layout?: "none" | "vertical" | "horizontal";
gap?: NumberOrVariable;
padding?: number | [vertical, horizontal] | [top, right, bottom, left];
justifyContent?: "start" | "center" | "end" | "space_between" | "space_around";
alignItems?: "start" | "center" | "end";
```

并且 `SizingBehavior` 支持类似 `fit_content` 和 `fill_container` 的动态尺寸。

🔒 Pencil +1

这个设计很重要，因为它避开了纯 Figma 式自由绝对定位的失控问题，同时又保留画布编辑自由度。

5. Pencil.dev 的组件逻辑： `reusable: true` + `ref` + `descendants`

Pencil 的组件不是 React 组件，而是 `.pen` object tree 里的可复用子树。一个对象设置 `reusable: true` 后就成为可复用组件；实例通过 `type: "ref"` 和 `ref: "component-id"` 引用该组件。

🔒 Pencil

基础形态是：

❏ JSON



```
{
  "id": "button",
  "type": "frame",
  "reusable": true,
  "children": [
    {
      "id": "label",
      "type": "text",
      "content": "Submit"
    }
  ]
}
```

实例是：

❏ JSON



```
{
  "id": "button-instance-1",
  "type": "ref",
```

```
"ref": "button"
}
```

更关键的是，Pencil 支持实例覆盖。实例可以直接覆盖组件根节点属性，比如 `fill`；也可以通过 `descendants` 覆盖组件内部子节点，比如把 `button` 里的 `label` 文案改掉。官方文档里还说明，嵌套实例的子节点可以用类似 `ok-button/label` 的 `path` 方式覆盖。

 Pencil +1

这就是 Pencil.dev 最值得你参考的地方之一：

```
Component Origin
  → Ref Instance
  → Root-level overrides
  → Descendant-level overrides
  → Descendant replacement
  → Children replacement
```

对 AI 设计来说，这比“让 AI 随便画新节点”强太多，因为它天然支持：

```
复用
覆盖
局部修改
批量替换
组件实例追踪
设计系统一致性
```

但它也有一个明显限制：**Pencil 的组件是视觉对象树组件，不一定是强语义业务组件。**比如它知道这是一个 `frame` 可复用组件，但未必天然知道它是 `ProductCard`、`PricingTable`、`DataTable`，也未必有严格 `typed props`。你要做自己的 AI 产品时，最好在 Pencil 这种视觉组件机制之上再加一层 **Semantic Component Manifest**。

6. Slot 逻辑：Pencil 已经有“AI 可理解组件组合”的雏形

Pencil 的 slot 是组件内部可被填充的区域。官方文档说，slot 是 component 内指定的可 drop 区域，用来定义灵活、可定制的组件区域。  Pencil

在 `.pen` schema 里，slot 是 `frame` 上的属性：

```
</> JSON
{
  "id": "content",
  "type": "frame",
```

```
"slot": ["round-button", "icon-button"]
}
```

官方解释是，`slot` 数组里的条目是推荐填入该 slot 的 reusable child component；Pencil 会用特殊视觉效果显示 slot，并允许用户一键插入推荐组件。  Pencil

这对你要做的组件库系统非常关键。你真正需要的不是简单组件库，而是：

组件

- └ props
- └ variants
- └ slots
- └ allowed children
- └ allowed replacements
- └ allowed overrides
- └ AI affordances



Pencil 的 slot 是正确方向，但你可以做得更强：Pencil 的 slot 更像“推荐”；你的系统应该把它升级成 **强类型 slot contract**。

例如：

YAML



```
component: ui.pricing-table
slots:
  plans:
    accepts: [ui.pricing-plan-card]
    min: 2
    max: 4
    layout: horizontal
  cta:
    accepts: [ui.button]
    max: 1
    required: true
```

这样 AI 不能乱塞东西。它只能在 contract 允许范围内组合组件。

7. Variables / Themes：Pencil 的 token 系统是 AI 设计一致性的基础

Pencil 的 variables 类似 CSS custom properties 或 design tokens：颜色用 HEX、间距/圆角/尺寸用数字、字体用字符串；变量一处修改，全局使用处更新。官方文档也明确说可以从 CSS 里让 AI agent 提取变量，也可以从 Figma token 表里导入，并支持 light/dark 这类 theme 列。  Pencil

`.pen` 格式支持 document-wide variables，例如：

`</>` JSON



```
{
  "variables": {
    "color.background": {
      "type": "color",
      "value": "#FFFFFF"
    },
    "color.text": {
      "type": "color",
      "value": "#333333"
    },
    "text.title": {
      "type": "number",
      "value": 72
    }
  }
}
```

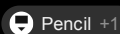
节点属性可以引用变量：

`</>` JSON



```
{
  "id": "welcome-label",
  "type": "text",
  "fill": "$color.text",
  "fontSize": "$text.title"
}
```

Pencil 还支持 theme axes，例如 `mode: light/dark`、`spacing: regular/condensed`，变量值可以根据 theme 条件动态解析。



这个点对你的产品几乎是必选项。你要做 AI 设计系统，必须把 token 做成一等公民：

AI 不应该说：把按钮颜色改成 #2563EB
AI 应该说：把按钮 fill 绑定到 \$color.primary



更进一步，你自己的系统应该区分：

raw value	# #2563EB
semantic token	# color.action.primary
component token	# button.primary.background
state token	# button.primary.hover.background
theme condition	# dark / light / compact / spacious



Pencil 已经有 variables/themes 的方向，但如果你要做“企业级完全可控”，需要比它更严格地做 token provenance、token lint 和 usage policy。

8. Code on Canvas: Pencil 的“可编程组件”机制非常值得抄

Pencil 有一个非常重要的能力叫 **Code on Canvas**。它通过 Script node 把 JS 文件挂到画布上：script 输出会渲染成 nested layers，适合图表、网格、装饰图案、数据驱动结构、重复变化元素、可参数化组件。 

一个最小 script 长这样：

 JavaScript 

```
/**
 * @schema 2.11
 *
 * @input columns: number(min=1) = 3
 * @input color: color = #3B82F6
 */
const cols = Math.floor(pencil.input.columns);
const cellW = pencil.width / cols;
const nodes = [];

for (let c = 0; c < cols; c++) {
  nodes.push({
    type: "rectangle",
    x: c * cellW,
    y: 0,
    width: cellW - 4,
    height: pencil.height,
    fill: pencil.input.color
  });
}

return nodes;
```

Script 的 `@input` 会自动变成 properties panel 控件，支持 number、string、boolean、color、enum、ref 等输入类型；改输入会重新运行 script；脚本输出是同一 `.pen` schema 的 node array。 

更重要的是它的安全和确定性设计：

`.js` 文件在 `.pen` 外部
script node 只存 path 和 input values
输出是 derived state



```
脚本运行在 sandbox
不能访问 DOM / network / filesystem
必须同步运行
Math.random() deterministic
最多返回 1000 nodes
必须 2 秒内完成
```

这些规则官方文档都有写。  Pencil

这对你的产品启发很大。你可以把它升级成：

```
Composable Generator Component
├ typed inputs
├ deterministic output
├ sandbox runtime
├ token binding
├ component refs as input
├ output validation
└ convert-to-static snapshot
```

比如：

```
chart.generator
pricing-table.generator
dashboard-layout.generator
form-builder.generator
calendar.generator
data-table.generator
```

这类东西让 AI 不必一个个画节点，而是调用受控生成器：

```
create DataTable with 6 columns, compact density, sortable header
```

然后系统根据 schema 生成一棵合法 node tree。

9. MCP 层：Pencil 的 AI 控制面真正厉害在这里

Pencil 通过 MCP 让 AI assistant 读写 `.pen`。官方解释是：MCP 是给 AI assistant 操作设计文件的 API，AI 可以通过工具读取、修改、生成设计；Pencil MCP Server 本地运行，设计操作没有云依赖；AI suggest，用户 approve。  Pencil

官方列出的 MCP 工具非常关键：

```
batch_design
- create / modify / manipulate design elements
- insert / copy / update / replace / move / delete
```

```
- generate and place images

batch_get
- read design components and hierarchy
- search elements by patterns
- inspect component structure

get_screenshot
- render design previews
- verify visual output
- compare before / after

snapshot_layout
- analyze layout structure
- detect positioning issues
- find overlapping elements

get_editor_state
- current editor context
- selection information
- active file details

get_variables / set_variables
- read design tokens
- update theme values
- sync with CSS
```

这些都来自官方 AI Integration 文档。  Pencil

这说明 Pencil 的 AI 流程不是：

LLM 直接输出最终设计文件 

而是：

```
LLM
→ get_editor_state
→ batch_get
→ get_variables
→ batch_design
→ snapshot_layout
→ get_screenshot
→ 修复
→ 用户确认 
```

这就是你产品应该重点学的“AI 可控设计”核心。

但你也要看到它的风险：`batch_design` 是一个很强的通用写工具。它很灵活，但如果你要“完全可操控的组件库”，你最好不要只提供一个万能 `batch_design`，而应该提供更强类

型、更语义化的 tool surface：

```
component.insert
component.set_variant
component.set_prop
component.fill_slot
component.replace_slot_child
token.set
theme.switch
layout.snapshot
layout.fix_overflow
code.sync_component
qa.check
```

万能工具给 AI 自由；强类型工具给产品可控性。

10. CLI 层：Pencil 已经把设计操作做成了自动化引擎

Pencil CLI 是 standalone command-line tool，可以从终端创建和编辑 `.pen` 文件。官方说它运行和 desktop app / IDE extension 相同的 editor engine，并且可以完全 headless，无 GUI。  Pencil

CLI 支持：

```
pencil --out design.pen --prompt "Create a login page..."
pencil --in existing.pen --out modified.pen --prompt "Add a blue submit button"
pencil --in design.pen --export design.png
pencil interactive -o design.pen
```

并且 agent mode 支持输入/输出 `.pen`、prompt、model、workspace、export、batch tasks 等参数。  Pencil

Interactive mode 更值得注意：它允许直接调用 MCP tools，对 `.pen` 做细粒度调试和脚本化操作；可以连接正在运行的 desktop/VSCode，也可以 headless 启动本地 editor；官方示例里直接调用 `get_editor_state({ include_schema: true })`、`get_guidelines()`、`batch_design(...)`、`get_screenshot(...)`、`save()`。  Pencil +1

这对你产品的启发是：设计系统不要只做 GUI，要从第一天就有 headless engine。

你的系统应该有：

```
GUI Editor
IDE Extension
MCP Server
CLI
```

```
CI/CD mode
Batch design tests
Screenshot regression
Design lint
Token sync check
Component migration
```

这样 AI 设计才会进入工程体系，而不是停留在“玩具画布”。

11. Design ↔ Code: Pencil 的双向同步逻辑

Pencil 的 Design ↔ Code 文档明确支持两条路：

```
Design → Code: 从 Pencil designs 生成 components
Code → Design: 把已有 code import 到 Pencil
```



它的 basic export workflow 是：先在 Pencil canvas 里设计 screen/layout/component，把 `.pen` 存在项目 workspace，打开 AI chat，让 Pencil 生成代码。示例 prompt 包括生成 React component、TypeScript types、Next.js page、Tailwind CSS、shadcn/ui、React Hook Form、Lucide icons 等。  Pencil

Code → Design 的逻辑是：如果已有组件在代码库里，Pencil 可以 visually recreate 它们；要求是 `.pen` 和代码在同一个 workspace，AI agent 能访问两边文件；导入内容包括 component structure/hierarchy、layout/positioning、styling。  Pencil

Two-way sync 的推荐流程是：

1. 从代码开始，把现有组件导入 Pencil
2. 在 Pencil 里做视觉改进
3. 让 AI 把变化应用回代码
4. 反复迭代



官方也建议把 `.pen` 放在 repo 中，让 AI agent 同时看到 design 和 code，并一起被 version control 管理。  Pencil

这对你的产品非常重要，但也有一个隐含问题：AI 负责 design-code sync，未必等于系统层面保证 design-code sync。

如果你要做更强的产品，不能只靠 prompt 说“同步一下”。你需要建立三层映射：

```
Semantic Component Manifest
↔ Visual Design Graph
↔ Code Component AST / Source Files
```



例如：

</> YAML

```
component: ui.button
design:
  penComponentId: "button"
  editableDescendants:
    label: content
    icon: icon
code:
  package: "@acme/ui"
  import: "Button"
  props:
    label: children
    variant: variant
    size: size
tokens:
  background: "button.primary.background"
```

这样 AI 才知道：改了 `.pen` 里的 `button/label.content`，代码里应该改 `children`；改了 `fill: "$color.primary"`，代码里应该映射到 `className="bg-primary"` 或 CSS var，而不是随机写死颜色。

12. 和 OpenDesign 的正确组合方式

你前面说 OpenDesign 查对了，这个判断可以保留。但现在应该重新定义二者关系：

维度	pencil.dev	OpenDesign
核心对象	<code>.pen</code> 设计文件 / object tree / canvas	artifact / skill / DESIGN.md / agent workspace
AI 控制方式	MCP tools 直接读写设计画布	agent 根据 skill 生成文件和预览
文件模型	JSON <code>.pen</code> ，可 Git 管理	artifact folder，HTML/JSX/PPTX 等文件
设计内核	画布、组件、ref、slot、变量、script node	模式、skill、agent adapter、preview/export
更适合学什么	AI 如何控制设计图	AI 如何组织多任务设计 workflow
不足	组件语义和 code binding 需要进一步加强	缺少强约束组件图内核

正确结论应该改成：

Pencil.dev 给你：

AI 可操作设计画布内核

OpenDesign 给你：

AI 任务编排和 artifact workflow

你真正要做的是：

Semantic Component Graph

+ Pencil-like Visual Graph

+ OpenDesign-like Skill/Agent Workflow

+ 强类型 MCP tools

13. 你要做的产品不应该直接复刻 Pencil，而应该升级 Pencil

Pencil 已经很接近你想要的方向，但如果你的目标是“完全可操控的基于组件库的 AI 设计”，我建议不要简单照搬 `.pen` 模型作为唯一数据源。

因为 Pencil 的 `.pen` 更偏 视觉对象树：

```
frame
rectangle
text
icon
ref
script
fill
stroke
layout
```

而你需要的是 语义组件图：

```
Page
Hero
Navbar
Button
PricingTable
FeatureGrid
DataTable
DashboardShell
Form
Input
Chart
```

我的建议是做双层 IR：

```
Semantic Graph    # 业务/组件/AI 操作层
  ↓ compile
Visual Graph      # Pencil-like canvas/render layer
  ↓ render/export
Code Graph        # React/Vue/HTML/CSS/Native implementation
```

也就是：

```
AI 不直接操作 visual node
AI 操作 semantic component node
系统把 semantic node 编译成 visual node 和 code
```

这样比 Pencil 更可控。

14. 推荐你的核心数据模型

14.1 Semantic Component Manifest

每个组件都要有一个 manifest，不只是 React 组件说明，而是 AI 可操作协议：

</> YAML

```
id: ui.hero
version: 1.0.0
displayName: Hero Section
category: marketing

props:
  title:
    type: string
    required: true
    maxLength: 80
    mapsTo:
      visual: "headline.content"
      code: "title"
  subtitle:
    type: string
    maxLength: 180
    mapsTo:
      visual: "subtitle.content"
      code: "subtitle"
  variant:
    type: enum
    values: ["centered", "split", "editorial"]
```

```

slots:
  actions:
    accepts: ["ui.button"]
    min: 1
    max: 2
  media:
    accepts: ["ui.image", "ui.video", "ui.illustration"]
    max: 1

tokens:
  required:
    - color.background
    - color.text.primary
    - typography.display
    - spacing.section.y

visualTemplate:
  type: pen-like
  componentId: "hero-origin"
  overrideMap:
    title: "headline.content"
    subtitle: "subtitle.content"
    background: "root.fill"

codeBinding:
  framework: react
  import: "@components/Hero"
  props:
    title: "title"
    subtitle: "subtitle"
    variant: "variant"

ai:
  allowedOps:
    - set_prop
    - set_variant
    - fill_slot
    - reorder_slot_children
  forbiddenOps:
    - arbitrary_absolute_position
    - hardcoded_color
    - replace_root_with_raw_frame

```

Pencil 的 `reusable/ref/descendants/slot` 可以作为 visual template 层，但你的 semantic manifest 要放在它上面。

14.2 Design Graph

你的源数据应该长这样：

</> JSON



```
{
  "version": "0.1",
  "tokens": "tokens.json",
  "components": "components.lock.json",
  "pages": [
    {
      "id": "home",
      "type": "page",
      "children": [
        {
          "id": "hero_1",
          "component": "ui.hero",
          "variant": "split",
          "props": {
            "title": "Design on canvas. Land in code.",
            "subtitle": "Build product screens with your real component system.",
            "density": "spacious"
          },
          "slots": {
            "actions": [
              {
                "id": "cta_primary",
                "component": "ui.button",
                "variant": "primary",
                "props": {
                  "label": "Get started"
                }
              },
              {
                "id": "cta_secondary",
                "component": "ui.button",
                "variant": "secondary",
                "props": {
                  "label": "Watch demo"
                }
              }
            ]
          }
        }
      ]
    }
  ]
}
```

然后系统编译成 Pencil-like visual graph:

</> JSON



```
{
  "version": "2.13",
  "variables": {},
  "children": [
    {
      "id": "hero_1_visual",
      "type": "ref",
      "ref": "hero-origin",
      "descendants": {
        "headline": {
          "content": "Design on canvas. Land in code."
        },
        "subtitle": {
          "content": "Build product screens with your real component system."
        }
      }
    }
  ]
}
```

这就是你和 Pencil 的关键差异：

Pencil 的 ref 是视觉复用；你的 graph 是语义复用。

15. 你的 MCP / Tool 设计应该比 Pencil 更强类型

Pencil 提供 `batch_design`、`batch_get` 这种大工具，非常灵活。你可以学习它，但产品化时最好拆成更安全的工具层。

建议你提供这些 MCP tools：

```
component_registry.search
component_registry.get_manifest
design_graph.get
design_graph.apply_ops
design_graph.validate
design_graph.diff
component.insert
component.set_prop
component.set_variant
component.fill_slot
component.replace_slot_child
component.detach_instance
token.get
token.set
theme.switch
layout.snapshot
layout.check_overflow
visual.render_screenshot
```



```
visual.compare_screenshot
code.import_component
code.sync_component
qa.check_accessibility
qa.check_design_rules
```

AI 修改设计时，只能返回 operation：

JSON

```
[
  {
    "op": "component.set_prop",
    "nodeId": "hero_1",
    "prop": "title",
    "value": "Build faster with AI-native design components"
  },
  {
    "op": "component.set_variant",
    "nodeId": "hero_1",
    "variant": "editorial"
  },
  {
    "op": "component.fill_slot",
    "nodeId": "hero_1",
    "slot": "actions",
    "children": [
      {
        "component": "ui.button",
        "variant": "primary",
        "props": {
          "label": "Start designing"
        }
      }
    ]
  }
]
```

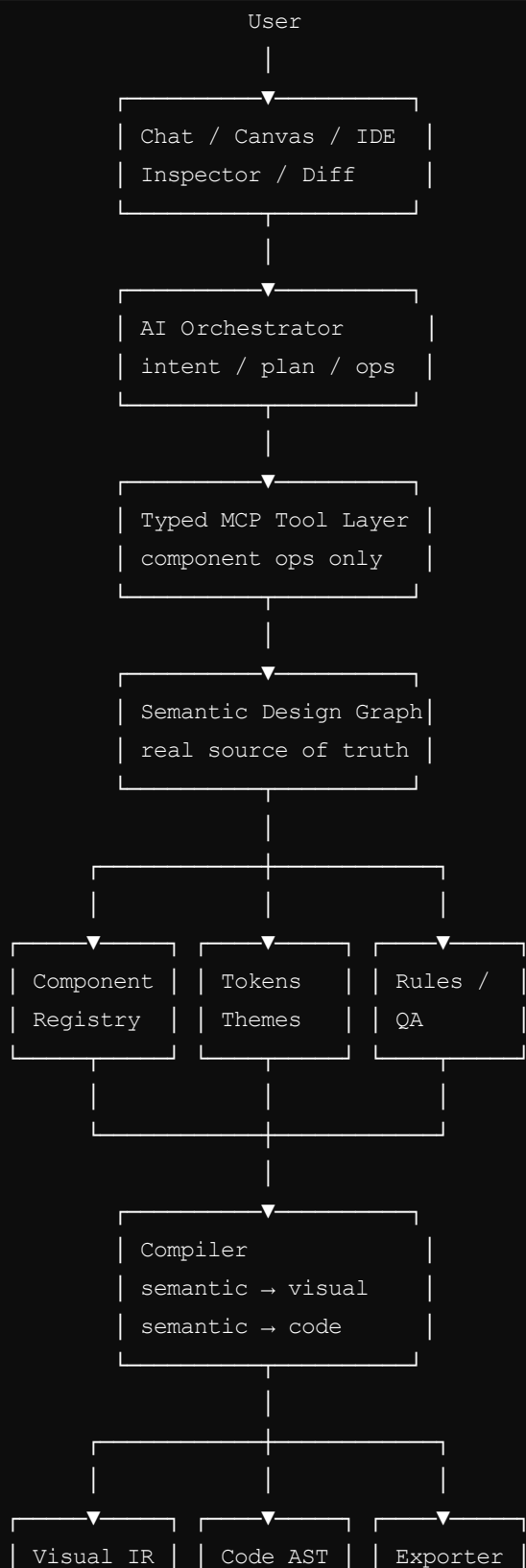
然后系统做：

1. schema validation
2. component manifest validation
3. slot contract validation
4. token policy validation
5. layout validation
6. screenshot render
7. visual QA
8. diff preview
9. apply / reject

这就是“完全可控”的关键。不是让 AI 变聪明，而是让 AI 只能走你的设计操作协议。

16. 产品架构建议：Pencil.dev + OpenDesign 的升级版

我建议你设计成下面这种架构：



| pen-like | | React/Vue | | PNG/PDF |

这一版的核心思想是：

```
Pencil.dev-like canvas
但 source of truth 不是视觉节点
而是 semantic component graph
```



这会让你比 Pencil 更适合做企业组件库 AI 设计。

17. 你应该重点学习 Pencil.dev 的 6 个设计选择

第一，设计文件进 repo

Pencil 强调 `.pen` 文件可以像代码一样 commit、diff、branch、merge。  Pencil

你也必须这样做：

```
design.graph.json
tokens.json
components.lock.json
visual.pen.json
code.bindings.json
```



全部进入 Git。这样 AI 设计才可审计、可回滚、可协作。

第二，本地 MCP Server

Pencil 的 MCP server 本地运行，AI assistant 连接后可读写 `.pen`。  Pencil

你也应该做本地优先：

```
localhost MCP server
local project files
local render engine
optional cloud models
```



这样企业客户会更容易接受，因为设计系统和源码不必全部上传到你的云端。

第三，读工具和写工具分离

Pencil 有 `batch_get`、`get_editor_state`、`snapshot_layout`、`get_screenshot` 这类读/分析工具，也有 `batch_design`、`set_variables` 这类写工具。  Pencil +1

你的工具协议也必须分成：

```
read
analyze
propose
apply
verify
export
```



千万不要只有一个“生成设计”接口。

第四，组件实例支持 descendant override

Pencil 的 `descendants` 机制非常适合 AI 做局部编辑，因为它让实例可以保留组件结构，只改内部某个 id path。  Pencil +1

你应该保留这个思想，但加上语义映射：

```
semantic prop: button.label
  → visual descendant: label.content
  → code prop: children
```



第五，slot 给 AI 组合边界

Pencil 的 slot 和 suggested components 已经在告诉 AI：这个区域应该放什么。  Pencil +1

你要把它升级成硬约束：

```
slot accepts
slot min/max
slot layout
slot semantic role
slot replacement policy
```



第六，script node 让复杂设计可参数化

Pencil 的 Code on Canvas 很适合数据驱动和参数化设计，script 输入能自动变成控制面板，输出同 schema node array，运行在 sandbox 且 deterministic。  Pencil

你的系统可以用这个思路做高级组件：

```
ChartGenerator
DataGridGenerator
TimelineGenerator
CalendarGenerator
FormGenerator
DashboardGenerator
```



这会比纯手动画布更适合 AI。

18. Pencil.dev 目前对你来说的不足

站在你的目标看，Pencil.dev 不是终点，它是很好的参考对象，但你可以做得更强。

不足 1：视觉组件强，语义组件不够强

Pencil 的 component 是 object tree 组件。它知道 reusable/ref/slot，但不一定有完整的业务语义、typed props、code binding 和 design rule。

你要做的是：

```
Button 不只是 frame + text
Button 是一个有 props、variant、state、slot、token、code binding 的 semantic component
```



不足 2：batch_design 太通用

通用工具灵活，但 AI 容易越界。你的系统应该提供更窄、更强类型的 tool。

```
差：batch_design("随便插一些 frame")
好：component.insert({ component: "ui.pricingTable", variant: "3-tier" })
```



不足 3：design-code sync 过度依赖 agent 推理

Pencil 的设计和代码同步很强，但从官方文档看，很多同步动作是通过 AI prompt 发起，例如“Update globals.css with these Pencil variables”或“Recreate Button component from src/components/Button.tsx”。  Pencil +1

你应该把它工程化成 binding 系统：

```
component manifest
prop map
token map
AST map
visual node map
migration rule
```



不足 4：组件库治理还可以更系统

Pencil 有 design library，`.lib.pen` 可以导入，库中组件更新会反映到使用处。  Pencil

但你如果做企业组件库，需要：

```
component versioning
deprecation
migration
compatibility matrix
team ownership
usage analytics
approval workflow
visual regression
breaking-change detection
```



19. 你的 MVP 应该怎么做

我建议你别一开始做完整 Figma，也别直接做 OpenDesign 那种多 artifact 大平台。先做一个 **Pencil.dev-like** 但更组件语义化的 **AI Design Kernel**。

MVP 目标

```
用自然语言基于组件库生成和修改 Web 产品界面
但 AI 只能通过 typed component operations 改 Design Graph
```



第一阶段：组件协议

先做 12 个核心组件：

```
ui.page
ui.section
ui.stack
ui.grid
ui.navbar
ui.hero
```



```
ui.button
ui.card
ui.feature-grid
ui.pricing-table
ui.form
ui.footer
```

每个组件都有：

```
props schema
variant schema
slot schema
token usage
layout behavior
visual template
code binding
AI allowed operations
QA rules
```

第二阶段：Graph + Renderer

实现：

```
Semantic Design Graph
  → Visual Canvas Preview
  → React Renderer
  → Static HTML Renderer
```

先不要做太复杂的无限画布。你可以先用 section-based editor，后面再拓展到自由画布。

第三阶段：MCP Tools

提供工具：

```
get_component_registry
get_design_graph
apply_design_ops
validate_design_graph
render_screenshot
snapshot_layout
check_design_rules
sync_to_code
```

AI 的写入必须是：

</> JSON



```
{
  "ops": [
    {
      "type": "set_prop",
      "nodeId": "hero_1",
      "prop": "title",
      "value": "Ship polished interfaces with AI"
    }
  ]
}
```

不要让 AI 直接写 HTML。

第四阶段：Inspector + Diff

每次 AI 改设计都展示：

新增了哪些组件
删除了哪些组件
改了哪些 props
改了哪些 tokens
有没有违反 slot contract
有没有产生 layout overflow
生成前后截图 diff
是否同步代码



这会形成产品信任感。

第五阶段：Code Sync

先支持一种栈：

React + TypeScript + Tailwind



然后扩展：

Next.js
Vue
Svelte
React Native
Flutter



但底层不要绑定某个框架。组件 manifest 要框架无关，renderer/code binding 再做 framework-specific。

20. 最终判断

正确版本应该是：

Pencil.dev 的核心价值：

```
.pen open JSON design format
local MCP read/write tools
reusable/ref/descendant override
slot-based component composition
variables/themes
script node programmable design
IDE + desktop + headless CLI same engine
design ↔ code workflow
```

OpenDesign 的核心价值：

```
skill/mode/artifact/agent
DESIGN.md 注入
多 agent adapter
preview/export pipeline
```

你应该做的：

```
用 Pencil.dev 的“AI 可操作设计画布”做底层参考
用 OpenDesign 的“AI workflow”编排“做设计”
但核心自研一个更强的 Semantic
```

哦，那你帮我查一下，就是基于 figma mcp 去读组件库，然后通过组件库的什么约束或者一些什么处理，让 pencil 能精准理解这个组件库的设计原则及使用原则，然后根据这一套原则，再用 pencil 做成实际的效果图。这几个流程里面，如果我们让这个组件库完全可控，让生成结果更加直接可靠（也就是不违背这个组件的设计原则）的话，我们需要怎么做？其实我原来粗浅的理解就是，我可能需要比如说通过 prompt 或者是什么什么一些数据库或者什么一些计算文件，把这个每一个组件的相关信息、使用方法，可能涉及到整体调节相关的东西。

一句话总结：

Pencil.dev 证明了 AI 设计工具的正确方向不是“生成图片”，而是“让 AI 通过 MCP 操作一个开放、可版本化、可渲染、可同步代码的设计 IR”。你要做的下一步，是把这个 IR 从视觉对象树升级成强类型语义组件图。你的方向是对的，但重点不是堆 prompt，而是把组件库抽象成 AI 可检索、可验证、可执行的组件契约。我会先核对 Figma MCP、Pencil 与组件变量能力，再拆成可落地流程。

我已确认：Figma MCP 更适合采集组件、变量、布局、Code Connect 等事实，Pencil 则适合承接为 `.pen` 组件、变量、slot、ref 和截图校验。关键是中间自研“组件契约编译层”，把映射变成系统约束，而不是只给模型阅读。

已思考 5m 49s >

你理解得基本是对的，但我会把它升级一下：

不要把目标定义成“让模型百分之百理解组件库”。

更好的目标是：即使模型理解错了，也不能提交违反组件规则的设计。

也就是说，你需要的不只是 prompt、数据库、映射文件，而是一套 **Design System Control Plane / 组件库控制层**。Figma MCP 负责把组件库读出来，Pencil 负责承载和生成 .pen 画布效果图，但中间必须有你自己的 **组件契约**、**token 映射**、**检索**、**编译**、**校验**、**纠错记忆**。

1. 先说结论：正确流程不是 Figma MCP → Prompt → Pencil

正确流程应该是：

```
Figma MCP / Figma API / Code Connect
↓
组件库抽取与归一化
↓
Component Contract / Token Graph / Usage Rules
↓
Pencil Library Compiler
↓
生成 .lib.pen / variables / reusable components / slots
↓
AI 只输出语义化设计操作
↓
Validator 校验
↓
编译成 Pencil batch_design 操作
↓
Pencil 生成效果图
↓
snapshot_layout + screenshot + rule check
↓
通过后保存 / 不通过则自动修复
```

最关键的是中间这一层：

```
Component Contract
Token Dependency Graph
Usage Rule Engine
Figma → Pencil Mapping
Pencil → Design System Validation
Correction Memory
```

你原来说的“给每个组件一个 AI 可以理解的完整映射”非常接近，但还要再补一层：这个映射不能只是给模型看的文档，它必须是机器可执行的规则。

2. Figma MCP 能给你什么，不能给你什么

Figma MCP 现在已经能提供非常有价值的组件库上下文。官方文档列出的工具包括

`get_design_context`、`get_metadata`、`get_screenshot`、`get_variable_defs`、`get_libraries`、`search_design_system`、`get_code_connect_map`、`get_context_for_code_connect`、`use_figma` 等；其中 `search_design_system` 可以搜索设计库里的组件、变量、样式，`get_variable_defs` 可以拿到选择范围使用的变量和样式，`get_context_for_code_connect` 可以返回组件属性、variants、descendant tree、property references 等结构化信息。  Figma 开发者文档

但 Figma 官方也说得很清楚：MCP 不是“一键完美设计到代码”的工具，它只是把 Figma 和 IDE/AI assistant 连接起来，提供结构化设计输入和代码起点；它不会默认理解你的设计系统，除非你通过 Code Connect、variables、prompt/rules 给它这些约定。  Figma 开发者文档

所以你不能指望：

Figma MCP 读一下组件库
→ 模型自然就懂了
→ Pencil 自然就能生成正确效果



你应该把 Figma MCP 当成 **抽取器**，不是当成 **真理层**。

3. Pencil 在这条链路里应该扮演什么角色

Pencil 很适合做最终的 AI 画布执行层。它的 `.pen` 文件本身是 JSON object tree，每个对象有 `id` 和 `type`，支持 frame、text、rectangle、script 等图形实体，也支持 flexbox-style layout；这让它很适合作为 AI 可写入、可版本化的设计 IR。  Pencil

Pencil 的组件机制也刚好适合你要做的“组件库可控”。Pencil 里对象可以通过 `reusable: true` 变成组件，通过 `type: "ref"` 创建实例；实例可以通过 `descendants` 覆盖组件内部子节点，也可以替换 children，非常适合做组件实例的局部覆盖。  Pencil

Pencil 还支持 slots。组件里的 frame 可以标成 slot，并声明推荐可放入的组件；官方文档也说 suggested slot components 会给人类和 AI 设计师提示“这个 slot 里应该放什么”。

 Pencil

变量和主题也很关键。Pencil variables 类似 CSS custom properties / design tokens，可以定义颜色、间距、圆角、尺寸、字体等，并在设计中引用；变量变化会更新所有使用位置，且支持 light/dark 这类 theme。  Pencil

所以 Pencil 这边最适合承接的是：

```
Figma component → Pencil reusable component
Figma instance → Pencil ref
Figma component properties → Pencil descendants / metadata / variant refs
Figma slots → Pencil slots
Figma variables / modes → Pencil variables / themes
Figma layout intent → Pencil frame layout
Figma design library → Pencil .lib.pen
```



但 Pencil 本身不应该是“组件语义理解层”。它知道 frame、ref、slot、variables，但它不一定知道这个组件在业务上是 `PricingCard`、`SearchBar`、`DataTableToolbar`，也不一定知道“这个 Button 在 Modal footer 里只能出现两个，primary CTA 只能一个”。这一层要你自己做。

4. 真正应该搭建的是：组件库控制层

我建议你吧系统分成 7 层。

```
1. Figma Ingestion Layer
2. Component Contract Layer
3. Token Graph Layer
4. Usage Rule Layer
5. Pencil Library Compiler
6. AI Operation Layer
7. Validation + Feedback Layer
```



下面逐层说。

5. 第一层：Figma Ingestion Layer

这层负责从 Figma 里读出组件库，而不是直接让模型自由总结。

建议数据来源有四个：

```
Figma MCP
Figma REST API
Code Connect
人工补充文档 / Storybook / 组件 README
```



Figma MCP 可以用来做交互式读取，比如搜索组件、读变量、读选中节点结构、拿截图、读 Code Connect 映射。Figma REST API 更适合做后台批量索引，例如拉取 team library 里已发布组件和样式的 metadata；Figma 的组件 API 明确支持获取 published components/styles 的 metadata，包括 component key、file key、node id、thumbnail、name、description、created/updated time 等。

变量这部分也建议不要只靠 prompt。Figma Variables REST API 支持查询、创建、更新、删除 variables；变量可以存储可复用值，并可用于设计属性和 prototyping actions。

 [Figma 开发者文档](#) 变量 endpoint 还能枚举本地变量、远程变量、variable collections、modes、valuesByMode，并且文件节点里有 `boundVariables` 可以追踪设计属性绑定了哪个变量。 [Figma 开发者文档](#)

抽取时你至少要拿这些信息：

Component:

- Figma component key / node id / name / description
- component set / variant properties
- component properties: text / boolean / instance swap / slot
- auto layout / constraints / padding / gap / sizing
- bound variables
- nested instances
- allowed slot content
- screenshots / thumbnails
- example usages

Token:

- variable collections
- modes: light / dark / brand / density
- aliases
- scopes
- bound properties
- code syntax if available

Code Connect:

- code component name
- import path
- prop mappings
- code snippets
- custom MCP instructions
- accessibility notes

Code Connect 非常重要。Figma 官方说，Code Connect 会把设计组件和代码组件连接起来；MCP 在处理带 Code Connect 的组件时，会把当前 Figma component property values、import statements、component usage snippet、自定义 instructions 等注入上下文。 [Figma 开发者文档](#) 这正好可以作为你组件契约的原始材料。

6. 第二层：Component Contract，不是 prompt

你原来说“给每个组件一个针对 AI 可以理解的完整映射”，我会把它正式命名为：

Component Contract

它不是自然语言文档，而是结构化 schema。

举个 Button 的例子：

</> YAML



```
id: ds.button
source:
  figma:
    componentKey: "abc123"
    componentSetNodeId: "12:34"
  pencil:
    reusableId: "button"
  code:
    component: "Button"
    import: "@/components/ui/button"

meaning:
  role: "primary user action / secondary action / destructive action"
  useWhen:
    - "triggering an action"
    - "submitting a form"
    - "navigating to an important flow"
  doNotUseWhen:
    - "pure text link is enough"
    - "large card itself is clickable"

props:
  variant:
    type: enum
    values: [primary, secondary, ghost, destructive]
    figmaProperty: "Variant"
    pencilMapping:
      kind: "variantRef"
      refPattern: "button/{variant}/{size}/{state}"
      codeProp: "variant"

  size:
    type: enum
    values: [sm, md, lg]
    figmaProperty: "Size"
    codeProp: "size"

  label:
    type: string
    maxLength: 28
    figmaProperty: "Label"
    pencilMapping:
      kind: "descendant"
      path: "label.content"
    codeProp: "children"
```

```
iconLeft:
  type: componentRef
  accepts: [ds.icon]
  optional: true
  figmaProperty: "Icon left"
  pencilMapping:
    kind: "slot"
    path: "icon-left"

slots:
  iconLeft:
    accepts: [ds.icon]
    max: 1
  iconRight:
    accepts: [ds.icon]
    max: 1

tokens:
  consumes:
    background: "button.{variant}.background"
    text: "button.{variant}.foreground"
    border: "button.{variant}.border"
    focusRing: "focus.ring"

rules:
  - "primary button: max 1 per action group"
  - "destructive button cannot be default CTA"
  - "disabled state cannot be used as primary path"
  - "icon-only button must have ariaLabel"
  - "label length <= 28"

allowedOps:
  - setProp(label)
  - setVariant(variant)
  - setSize(size)
  - fillSlot(iconLeft)
  - fillSlot(iconRight)

forbiddenOps:
  - hardcodedFill
  - arbitraryResizeText
  - detachInstance
  - replaceRootWithRawFrame
```

这就是你要的“AI 能理解组件意义”的核心。但要注意：意义不是靠一句 **prompt** 保存的，而是拆成 **props**、**slots**、**tokens**、**rules**、**allowedOps**、**forbiddenOps**。

7. 第三层：Token Graph，解决你说的整体色调、字体、暗色亮色关联

你提到的这个问题非常关键：

整体主体色调变了，组件配色哪部分要调整；整体字体大小变了，这部分怎么调整；暗色亮色、文本之间的关联。

这个不能放在每个组件的 prompt 里散着写。你需要一张 **Token Dependency Graph**。

它应该是这样的：

```
primitive token
  ↓
semantic token
  ↓
component token
  ↓
component visual property
  ↓
Pencil node property / code prop / CSS variable
```

例如：

YAML

```
tokens:
  primitive:
    blue.600: "#2563EB"
    gray.950: "#030712"

  semantic:
    color.brand.primary:
      value: "{blue.600}"
    color.text.default:
      light: "{gray.950}"
      dark: "{gray.100}"
    color.surface.default:
      light: "{white}"
      dark: "{gray.950}"

  component:
    button.primary.background:
      value: "{color.brand.primary}"
    button.primary.foreground:
      value: "{color.text.onBrand}"
    card.background:
      value: "{color.surface.default}"

bindings:
```

```
ds.button:
  root.fill: "button.{variant}.background"
  label.fill: "button.{variant}.foreground"
  focusRing.stroke: "focus.ring"

ds.card:
  root.fill: "card.background"
  title.fill: "color.text.default"
  body.fill: "color.text.muted"
```

这样当用户说：

把整体主色调调成紫色



AI 不应该去改每个按钮的 fill。它应该做：

</> JSON



```
{
  "op": "token.update",
  "token": "color.brand.primary",
  "value": "#7C3AED"
}
```

然后系统通过 token graph 推导：

```
button.primary.background 变
link.foreground 变
focus.ring 变
selected.tab.indicator 变
chart.accent.1 可能变
```



再通过 Pencil variables 更新 `.pen`。Pencil variables 本来就支持全局变量和 theme，变量变化会更新所有引用处。  Pencil Figma variables 也支持 modes，用于 light/dark/theme 这类上下文切换。  Figma 帮助中心

所以你要做的是：

```
Figma Variables / Modes
  ↓
normalized tokens.json
  ↓
token alias graph
  ↓
Pencil variables / themes
  ↓
component property bindings
```



不要让 AI 自己猜哪个颜色该改，系统应该知道。

8. 第四层：Usage Rules，要分成硬约束和软规则

组件库设计原则里有两种东西：

硬约束：违反就不能生成

软规则：可以提示、扣分、需要解释

比如：

</> YAML

```
hardRules:
  - Button.label.maxLength <= 28
  - Modal.footer.actions.max <= 2
  - Table.column.count <= 8 on mobile
  - Input.errorText required when state == error
  - IconButton.ariaLabel required
  - All fills must use tokens
  - Raw hex not allowed except approved illustration assets
  - Slot accepts only declared component types

softRules:
  - Prefer Card over raw Frame for grouped content
  - Prefer Stack/Grid layout over absolute positioning
  - Marketing pages should use one primary CTA above fold
  - Empty states should include title, description, action
  - Avoid more than 3 font sizes in one card
```

Figma 自己也建议为了更好输出，要使用 components、Code Connect、variables、清晰语义命名、Auto Layout、annotations 和 dev resources。  Figma 开发者文档 这些建议在你的系统里不能只是“建议”，而应该变成 lint rule。

也就是说：

```
Figma guideline → 机器规则
Design principle → validator
Component usage doc → allowed operations
视觉偏好 → scoring function
```

9. 第五层：Pencil Library Compiler

你最终需要把 Figma 组件库编译成 Pencil 可使用的 `.lib.pen`。

Pencil design library 是 `.lib.pen` 文件，里面是可复用组件；当 library 里的组件变更，使用它的地方也会反映更新。  Pencil 这正好可以承接 Figma 组件库。

建议编译策略：

```
Figma component set
  → Pencil reusable component group

Figma variant
  → Pencil reusable variant ref
或
  → same reusable component + metadata variant map

Figma component property
  → Pencil metadata + descendants override path

Figma slot
  → Pencil slot + suggested slot components

Figma variables
  → Pencil variables

Figma modes
  → Pencil themes

Figma instance example
  → Pencil example frame / fixture

Figma description / docs
  → Component Contract markdown summary
```

Pencil `.pen` schema 里已经有 `metadata` 字段，可以把你自己的组件语义 ID、Figma key、contract version、allowed override paths 等写进去。 `.pen` 对象本身有 `metadata?: { type: string; [key: string]: any }` 这样的扩展点。 

例如编译后一个 Pencil 组件可以带：

```
<> JSON

{
  "id": "ds.button.primary.md.default",
  "type": "frame",
  "reusable": true,
  "metadata": {
    "type": "design-system-component",
    "componentId": "ds.button",
    "contractVersion": "1.3.0",
    "figmaComponentKey": "abc123",
    "variant": {
```

```

    "variant": "primary",
    "size": "md",
    "state": "default"
  },
  "allowedDescendantOverrides": {
    "label.content": "props.label",
    "icon-left.children": "slots.iconLeft"
  }
},
"children": [
  {
    "id": "label",
    "type": "text",
    "content": "Button",
    "fill": "$button.primary.foreground"
  }
]
}

```

这样 Pencil 画布里的东西不只是图形，而是带有组件库身份和规则的图形。

10. 第六层：AI 不能直接操作 Pencil，必须操作你的语义层

这是“完全可控”的关键。

不要让模型直接调用：

```
batch_design: create random frames, texts, rectangles
```



Pencil MCP 的 `batch_design` 很强，可以创建、修改、复制、更新、替换、移动、删除设计元素；`batch_get` 可以读取组件和层级；`get_screenshot` 和 `snapshot_layout` 可以做视觉和布局验证。🔒 Pencil 但如果你把 `batch_design` 直接暴露给模型，模型仍然可能绕过组件规则，画一堆“看起来像组件”的 raw frame。

你要做一个中间 controller：

```

AI semantic ops
  ↓
validate
  ↓
compile
  ↓
Pencil batch_design

```



AI 只能输出：

</> JSON



```
[
  {
    "op": "component.insert",
    "component": "ds.hero",
    "variant": {
      "layout": "split",
      "tone": "premium"
    },
    "props": {
      "title": "Build interfaces with your real design system",
      "subtitle": "Generate reliable screens from governed components."
    },
    "slots": {
      "actions": [
        {
          "component": "ds.button",
          "variant": {
            "variant": "primary",
            "size": "lg"
          },
          "props": {
            "label": "Start designing"
          }
        }
      ]
    }
  }
]
```

它不能输出：

画一个 300x48 的蓝色圆角矩形，再放一段白色文字



因为那样已经绕过组件库了。

11. 第七层：Validation + Feedback，保证“不违背设计原则”

生成后必须做 5 类校验：

1. Schema validation
2. Component contract validation
3. Token validation
4. Layout validation
5. Visual validation



具体来说：

Schema validation:

- op 格式是否合法
- prop 类型是否合法
- enum 值是否合法

Component validation:

- 是否使用已注册组件
- variant 组合是否存在
- slot 里是否放了允许的组件
- 是否修改了禁止修改的 descendant

Token validation:

- 是否硬编码颜色
- 是否使用了合法 semantic token
- theme mode 是否完整
- dark/light contrast 是否达标

Layout validation:

- 是否使用 frame layout / auto layout
- 是否重叠
- 是否溢出
- mobile/tablet/desktop 是否满足规则

Visual validation:

- get_screenshot
- snapshot_layout
- compare against examples
- detect obvious spacing / alignment / contrast issue

Pencil MCP 的 `snapshot_layout` 本来就可以分析布局结构、检测定位问题和重叠元素；`get_screenshot` 可以渲染预览、验证视觉输出和比较前后效果。🖋️ Pencil 这两个应该成为你自动 QA 的固定步骤，而不是可选步骤。

12. 你说的“随用随建、建了存起来、也可以被纠正”是对的，但要换一种存法

这个思路非常好，尤其适合真实企业组件库，因为一开始不可能把所有组件都建得完美。

我建议叫：

Lazy Contract Building

流程是：

第一次遇到某个 Figma 组件

- 从 Figma MCP / Code Connect / screenshots / docs 抽取信息
- 生成 draft contract
- 根据实际使用场景补充 rules
- 生成 Pencil component mapping
- 运行 validator
- 人类确认 / 修改
- 保存为正式 contract

用户纠正时，不要只把纠正塞进对话历史，而要落到具体文件：

用户：这个组件在 dashboard 里不能用 primary variant，只能用 subtle。

↓

更新：

```
components/alert.contract.yaml
rules/contextual-usage.rules.yaml
examples/alert-dashboard-good.json
examples/alert-dashboard-bad.json
```

也就是说，纠错应该变成：

```
prompt memory ✗
contract patch ✔
rule patch ✔
example patch ✔
validator patch ✔
```

这样下一次无论换哪个模型、哪个上下文窗口，都不会忘。

13. 上下文窗口怎么省

你说“上下文窗口里尽量不占用太多，同时完全理解组件意义”，这个判断非常准确。解决方法不是压缩 prompt，而是做 分层检索。

不要把所有组件信息一次性塞给模型。应该分成 5 层：

```
L0: Global rules
L1: Component index
L2: Component summary card
L3: Full component contract
L4: Examples / screenshots / source snippets
```

运行时只给模型：

```
全局规则摘要
当前任务相关的 5~20 个组件摘要
```

当前选中组件的完整 contract
必要的 token graph 子图
必要的 examples

例如组件 index 可以很短：

</> JSON



```
{
  "ds.button": {
    "intent": "action trigger",
    "variants": ["primary", "secondary", "ghost", "destructive"],
    "slots": ["iconLeft", "iconRight"],
    "useFor": ["submit", "navigation CTA", "inline action"]
  },
  "ds.card": {
    "intent": "group related content",
    "slots": ["media", "header", "body", "footer"],
    "useFor": ["feature card", "pricing card", "settings group"]
  }
}
```

只有当模型真的要用 `ds.button`，才加载完整 contract。

这比“把每个组件写一大段说明放进 prompt”靠谱很多。

14. 数据库怎么设计

你可以同时用三种存储：

1. Git-tracked files
2. Relational / document DB
3. Vector index



我建议文件结构是：

```
design-system-control/
sources/
  figma.snapshot.json
  code-connect.snapshot.json

tokens/
  tokens.json
  token-alias-graph.json
  modes.json

components/
```



```
button.contract.yaml
card.contract.yaml
modal.contract.yaml
pricing-table.contract.yaml

mappings/
  figma-to-pencil.yaml
  pencil-to-code.yaml
  figma-to-code-connect.yaml

rules/
  global.rules.yaml
  accessibility.rules.yaml
  layout.rules.yaml
  brand.rules.yaml
  contextual-usage.rules.yaml

examples/
  button.good.json
  button.bad.json
  modal.good.json
  modal.bad.json

pencil/
  library.lib.pen
  fixtures.pen

generated/
  component-index.json
  retrieval-cards.json
  validator-schema.json
```

数据库里可以放：

```
component_id
figma_key
pencil_id
code_component
embedding
last_seen_at
contract_version
confidence_score
usage_count
corrections_count
```



vector index 只用来做语义检索，比如“我需要一个顶部导航里的搜索入口”能找到

`SearchInput`、`CommandButton`、`TopNav`。真正的规则判断不要靠 vector，相似度不能当约束。

15. 让 Pencil 精准理解组件库的关键不是“理解”，而是“编译”

这句话很重要：

Pencil 不需要自己理解 Figma 组件库。
你要把 Figma 组件库编译成 Pencil 能执行的组件库。

也就是：

```
Figma Button
→ Contract: ds.button
→ Pencil reusable component: ds.button/*
→ Pencil variables: $button.primary.background
→ Pencil slots: iconLeft, iconRight
→ Pencil descendants: label.content
→ Validator rules
```

这样 AI 在 Pencil 里生成效果图时，不是在“模拟 Figma 组件”，而是在使用你已经编译好的 Pencil 组件库。

16. 建议的实际生成流程

当用户说：

用我们的组件库做一个 SaaS 首页，主色调更偏紫色，高级一点。

系统应该这样跑：

```
Step 1: Intent parsing
- 页面类型: SaaS landing page
- 风格: premium
- token change: brand primary → purple
- 需要组件: navbar, hero, logo-cloud, feature-grid, testimonial, pricing, footer

Step 2: Retrieve contracts
- ds.navbar summary + full contract
- ds.hero full contract
- ds.button full contract
- ds.feature-grid summary
- token graph: brand.primary subtree

Step 3: Propose semantic ops
- token.update(color.brand.primary)
- component.insert(ds.navbar)
- component.insert(ds.hero)
```

- `component.fill_slot(hero.actions, ds.button)`
- `component.insert(ds.feature-grid)`

Step 4: Validate ops

- 是否所有组件存在
- 是否所有 slots 合法
- 是否 token 合法
- 是否 primary CTA 超量
- 是否移动端布局合法

Step 5: Compile to Pencil

- `set_variables`
- `batch_design insert refs`
- `descendants override` 文案
- `slot children replacement`

Step 6: Verify

- `snapshot_layout`
- `get_screenshot`
- `visual QA`
- `contrast check`
- `design rule check`

Step 7: Save

- `design.pen`
- `history.jsonl`
- `applied ops`
- `validation report`

这样生成结果会比单纯 prompt 稳定得多。

17. Prompt 还要不要？

要，但 prompt 只做三件事：

1. 任务意图解释
2. 高层设计方向
3. 工具使用流程



prompt 不应该承载所有组件知识。

你可以有一个很短的系统规则：

You are designing with a governed component library.



You must:

- search component registry before creating anything
- use semantic component operations only

- never create raw visual nodes unless explicitly allowed
- bind all colors, spacing, typography to tokens
- validate every operation before compiling to Pencil
- fix all hard rule violations before returning

真正的组件知识来自：

```
component contracts
token graph
rule engine
examples
retrieval cards
```



Figma 官方也建议用 rules / instructions 来减少重复提示，例如要求优先使用组件、避免硬编码、使用 tokens、遵守 WCAG、复用现有组件等；但这些官方示例更多还是“指导 AI”，你的系统应该进一步把它们变成 validator。  [Figma 开发者文档](#)

18. 推荐的“严格模式”

为了让组件库“完全可控”，建议设计三个模式：

Strict Mode

```
只能使用已注册组件
只能使用已注册 tokens
只能使用允许的 variants
只能填充允许的 slots
不能创建 raw frame/text/shape
不能 detach instance
不能 hardcode color / spacing / font size
所有变更必须通过 validator
```



适合企业设计系统、生产级 UI。

Guided Mode

```
优先使用组件库
允许创建 layout wrapper
允许轻微局部调整
但必须标记为 non-system node
最后进入 review
```



适合探索新页面结构。

Creative Mode

允许提出新组件草案
但不能混入正式组件库
生成结果进入 `candidate area`
需要设计师批准后才能变成 `contract`



适合探索新组件。

你的目标如果是“更加直接可靠”，默认应该是 **Strict Mode**。

19. 组件映射建议分成 6 类

你说的“小映射”可以分成下面几类，不要混在一起。

1. 语义映射

这个组件是什么？
什么时候用？
什么时候不用？
它在页面中的角色是什么？



例如：

`</>` YAML



```
ds.empty-state:
  role: "explain absence of content and guide next action"
  useWhen:
    - no data
    - no search results
    - permissions unavailable
  doNotUseWhen:
    - loading state
    - error state
```

2. 属性映射

Figma property ↔ AI prop ↔ Pencil descendant ↔ Code prop



例如：

`</>` YAML



```
label:
  figmaProperty: "Label"
  aiProp: "label"
```

```
pencilPath: "label.content"
codeProp: "children"
```

3. 变量映射

全局 token ↔ 组件 token ↔ 视觉属性



例如：

YAML



```
button.primary.background:
  source: "color.action.primary"
  pencil: "$button.primary.background"
  figmaVariable: "Button/Primary/Background"
  css: "--button-primary-bg"
```

4. slot 映射

哪个位置能放什么？
最多几个？
排列方式是什么？



例如：

YAML



```
modal.footer.actions:
  accepts: [ds.button]
  min: 1
  max: 2
  rules:
    - "primary max 1"
    - "destructive must be secondary unless destructive flow"
```

5. layout 映射

组件如何响应式？
padding/gap 怎么取 token？
移动端怎么重排？



例如：

YAML



```
ds.hero:
  desktop: "split"
  tablet: "stack"
  mobile: "stack"
  maxWidth: "$container.lg"
  paddingY: "$section.paddingY"
```

6. 纠错映射

用户纠正过什么?
哪类场景容易错?
下次如何避免?



例如：

</> YAML



```
corrections:
  - when: "dashboard filter bar"
    avoid: "primary button for every filter"
    prefer: "ghost button or select trigger"
    source: "user correction 2026-06-11"
```

20. 你这个方向最容易踩的坑

坑 1：把 prompt 当知识库

prompt 只能临时提醒，不能保证执行。真正可靠的是：

contract + validator + compiler



坑 2：让 AI 直接写 Pencil raw nodes

这样很快会出现“看起来像组件，但不是组件”的假组件。

必须让 AI 只写：

```
component.insert
component.set_prop
component.fill_slot
token.update
theme.switch
layout.apply_template
```



坑 3：只做向量数据库

向量数据库适合“找相关组件”，不适合“判断是否合法”。

合法性必须靠：

```
JSON Schema
Zod
rule engine
token graph
visual/layout checker
```



坑 4：把 Figma 变量直接平铺成 Pencil 变量

Figma 变量可能有 primitive、semantic、component、mode、alias。你要先归一化成 token graph，再编译到 Pencil variables。否则后面“整体主色调改变”会变成一堆局部颜色替换。

坑 5：没有版本

每个 contract 都要有版本：

```
</> YAML
```



```
component: ds.button
version: 1.3.0
figmaUpdatedAt: ...
pencilCompiledAt: ...
codeConnectVersion: ...
```

否则 Figma 组件变了，Pencil 里的库、AI 映射、代码映射会逐渐漂移。

21. 我建议你最终做成这个产品形态

```
Figma Component Library
  ↓
Design System Indexer
  ↓
Component Contracts + Token Graph
  ↓
Pencil Library Generator
  ↓
AI Design Runtime
  ↓
Pencil Canvas Output
  ↓
Validation Report + Screenshot Diff
```



产品能力可以这样包装：

1. Import from Figma
 - 读取组件库、变量、variants、slots、Code Connect
2. Generate AI Contracts
 - 自动生成组件说明、props、slot、token binding、usage rules
3. Review & Correct
 - 人类审核组件 contract
 - 修改后保存为长期规则
4. Compile to Pencil
 - 生成 .lib.pen
 - 生成 Pencil variables/themes
 - 生成 component fixtures
5. Design with AI
 - 用户自然语言描述
 - AI 只调用 semantic component ops
 - 系统编译到 Pencil
6. Validate
 - 组件合法性
 - token 合法性
 - layout 合法性
 - 视觉截图
 - 规则报告
7. Learn
 - 用户纠正转成 contract/rule/example patch



22. 一句话回答你“有没有更好的处理方式”

有，比“prompt + 数据库 + 计算文件”更好的方式是：

Prompt 负责流程，数据库负责检索，计算文件负责映射，但真正保证可靠的是：组件契约 + token 图谱 + 语义操作协议 + 编译器 + 校验器。

你原来的理解是对的，但还差最后一步：

让模型理解组件

我非常理解，那现在其实证明这件事情确实是可行的，也不算确实是可行的，就是理论上还行得通的。具体我们还要看这个模型跑出来的效果。

要升级成：

让系统强制模型只能合法使用组件

但我有另外一个疑问，就是我们有没有必要用 figma 到 pencil 这个步骤？还是说 figma 直接调用比如说 code x 写 HTML 页面，就跳过 pencil 直接出页面？



这样才会接近你想要的“百分之百不违背设计原则”。

但是我感觉可能应该是不如通过 pencil 做出来设计图的这种方法效率更高，然后 pencil 再做成 HTML，这种应该是更简单的。还有另外一个问题，就是你说的这些层确实是很

展开 ▾

我会先比较“直接 Figma/代码生成”和“保留 Pencil 中间层”两条路线，再给出一套用现成组件库快速验证受控生成效果的 MVP 流程和 prompt。

我查到的能力边界支持把 POC 做得很薄：同时跑直接 HTML 与 Pencil 中间层两条链路，优先验证已编译组件契约能否稳定约束 `.pen` 生成。

已思考 5m 34s >

你这个判断很对：现在不要做完整设计系统工程，也不要一开始就做 Figma→Pencil 的完整转换链路。

第一轮应该做一个很轻的 POC：证明“组件契约 + token 映射 + prompt 约束 + Pencil 画布验证”能不能显著提高 AI 生成结果的可控性。

我的建议是：

第一轮先跳过 Figma。

用 Material Design 或 Untitled UI 这种已有组件体系，先人工极少介入地让 AI 编译出一套“AI Design Kit”，再让 Codex / Claude Code / Pencil 去生成 `.pen` 和 HTML，对比两条路线效果。

1. 到底要不要 Figma → Pencil?

分两种情况。

如果目标是 尽快出一个可运行 HTML 页面，那可以直接：

组件库 / Figma / 设计说明

↓

Codex / Claude Code

↓

React / HTML 页面



这条路最快。尤其如果你的组件库已经有 React 代码、Code Connect 或明确的组件 API，直接生成 HTML/React 很合理。Figma 官方也把 Code Connect 定义成连接 Figma 组件和代码组件的关键能力，MCP 可以返回组件映射、源码位置、prop mapping、snippet 等信息；官方甚至说没有 Code Connect 时，模型更多是在猜。

 Figma 开发者文档 +2

但如果目标是 验证 AI 是否能稳定遵守组件库设计原则，我仍然建议加 Pencil，原因不是“Pencil 转 HTML 更厉害”，而是 Pencil 给你一个 中间设计验证层：

```
AI 生成意图
  ↓
.pencil 设计图
  ↓
截图 / 布局 / 组件实例 / token 使用验证
  ↓
再生成 HTML
```

Pencil 的 `.pen` 文件是 JSON object tree，有 frame、text、rectangle 等节点，支持 flexbox-style layout；它还有 reusable component、ref instance、descendant override、slot、variables、themes 等机制，正好适合做“AI 可操作设计中间层”。  Pencil +2

所以结论是：

```
第一轮 POC：
  不需要 Figma → Pencil。
```

```
真正产品化：
  如果用户的组件库源头在 Figma，就需要 Figma → Contract → Pencil。
  但不是 Figma → Pencil 一步到位，而是 Figma 先编译成组件契约。
```

2. 其实你要验证的不是 Pencil，而是“中间契约层”

第一轮最小验证应该是这个：

```
Material / Untitled / 任意已有组件库
  ↓
AI 编译成 AI Design Kit
  ↓
Codex / Claude Code 基于 AI Design Kit 生成 .pen
  ↓
Pencil 导出截图并检查
  ↓
同一套组件规则再生成 HTML
  ↓
和直接生成 HTML 对比
```

这里最关键的文件不是 `.pen`，而是这几个：

```
tokens.json
component-contracts/*.json
usage-rules.json
```

```
pencil-library-spec.json
AGENTS.md / CLAUDE.md
eval-tasks/*.json
```

也就是说，你不是先做一个完整设计系统，而是做一个很小的 组件库控制包。

3. Pencil 这一步的价值在哪里？

Pencil 的价值不是最终代码，而是 可视化、可验证、可纠错的设计 IR。

Pencil CLI 可以 headless 创建、编辑 `.pen`，也可以导出 PNG/JPEG/WEBP/PDF；它还支持在 interactive shell 里直接调用 MCP tools，比如 `batch_design`、`batch_get`、`get_variables`、`set_variables`、`snapshot_layout`、`get_screenshot`。

 Pencil +2

这意味着你可以让 AI 生成之后自动做：

1. 检查有没有 `raw frame` 冒充组件
2. 检查有没有硬编码颜色
3. 检查是否用了 `ref instance`
4. 检查 `slot` 里是不是放了允许组件
5. 检查 `layout` 有没有 `overlap / overflow`
6. 截图给人看
7. 不合格就自动修复



直接 HTML 路线也能做检查，但 HTML/CSS 的自由度更高，模型更容易绕过组件规则，比如：

看起来像 `Button`，但不是 `Button`
看起来像 `Card`，但没有用 `Card` 组件
颜色接近主色，但不是 `token`
间距看着差不多，但不是 `spacing scale`



Pencil 路线的好处是你可以把“有没有真的用组件库”检查得更直观、更结构化。

4. 但我不建议最终做 Pencil → HTML 单向转换

这里要稍微纠正一下你的直觉。

你说：

Pencil 做出来设计图，再做成 HTML，应该更简单



对 视觉原型 HTML 来说，这可能是对的。

但对 生产级 HTML / React 页面 来说，更好的方式不是：

```
Pencil .pen → 反推 HTML
```



而是：

```
Semantic Design Ops
```

```
├─ 编译成 Pencil .pen
```

```
└─ 编译成 React / HTML
```



也就是说，`.pen` 和 HTML 应该是同一个语义设计操作的两个输出，而不是 HTML 从 `.pen` 里硬抠出来。

更稳的结构是：

```
用户需求
```

```
↓
```

```
AI 生成 semantic ops
```

```
↓
```

```
组件契约校验
```

```
↓
```

```
输出 A: Pencil .pen 设计图
```

```
↓
```

```
输出 B: React / HTML 页面
```



这样你不会丢失语义。比如 `Hero`、`Button`、`PricingCard` 在 `.pen` 里是视觉组件，在 React 里是代码组件，但它们都来自同一份 `component-contracts`。

5. 第一轮 POC 最好选什么组件库？

我建议两个候选。

方案 A：Material Design / Material Web

优点是公开、标准化、token 理念成熟、资料容易拿。Material Design 3 官方把系统拆成 foundations、styles、components，Material Design tokens 用来跨设计和开发流程表达颜色、字体等设计决策；Material Web 是 Google 的 Material 3 Web Components 库。

 Material Design +2

更重要的是，Material 的 token 结构适合验证你说的：

```
整体主色变化
```

```
暗色 / 亮色变化
```

```
字体大小变化
```



spacing / radius / elevation 变化
组件属性跟 token 的映射

所以 Material 适合做 可复现 POC。

方案 B: Untitled UI

Untitled UI 更接近真实 SaaS 产品审美，而且有 Figma UI kit、React component library、icons，官方也强调它有大量组件、变量、dark mode variables、Figma 与 React 资源。

Untitled UI +1

但 Untitled UI 的完整 Figma/React 资源涉及授权。你如果已经买了或有访问权，它很适合做第二个 POC，因为它比 Material 更像你最终想生成的商业页面。

我的建议：

第一轮: Material-lite
第二轮: Untitled-lite
第三轮: 接真实客户 Figma 组件库



6. 第一轮 POC 不要覆盖完整组件库

只做一个很小的组件集：

Button
Card
TextField / Input
TopNav / Navbar
Hero
FeatureCard
PricingCard
Section
Footer



严格一点，第一轮甚至可以只做：

Button
Card
Hero
FeatureGrid
PricingCard



目标不是覆盖率，而是验证：

AI 是否真的复用组件
AI 是否遵守 props / variants



AI 是否使用 tokens
AI 是否能处理主题变化
AI 是否能通过 Pencil 验证
AI 生成 HTML 时是否仍然遵守同一套契约

7. 建议的 POC 文件结构

可以建一个这样的 repo:

```
ai-design-poc/  
  AGENTS.md  
  README.md  
  
  design-system/  
    tokens.json  
    token-map.json  
    usage-rules.json  
  
    components/  
      button.contract.json  
      card.contract.json  
      hero.contract.json  
      feature-card.contract.json  
      pricing-card.contract.json  
  
    retrieval-cards.json  
    pencil-library-spec.json  
    html-rendering-rules.json  
  
  pencil/  
    material-lite.lib.pen  
    landing-page.pen  
    dashboard-page.pen  
    exports/  
      landing-page.png  
      dashboard-page.png  
  
  html/  
    direct-html/  
    from-contracts/  
  
  evals/  
    task-01-saas-landing.json  
    task-02-pricing-page.json  
    task-03-theme-change.json  
    scorecard.md
```

这里面真正重要的是:

```
component.contract.json
tokens.json
usage-rules.json
pencil-library-spec.json
AGENTS.md
evals/*.json
```



`.pen` 可以让 AI 生成；HTML 也可以让 AI 生成；但契约文件必须成为它们共同的约束。

8. Component Contract 示例

第一轮不用复杂，做成这样就够了：

<> JSON



```
{
  "id": "ds.button",
  "name": "Button",
  "role": "Triggers a primary, secondary, or destructive user action.",
  "variants": {
    "intent": ["primary", "secondary", "tertiary", "destructive"],
    "size": ["sm", "md", "lg"],
    "state": ["default", "hover", "disabled"]
  },
  "props": {
    "label": {
      "type": "string",
      "required": true,
      "maxLength": 28
    },
    "leadingIcon": {
      "type": "component",
      "accepts": ["ds.icon"],
      "required": false
    }
  },
  "tokens": {
    "background": "button.{intent}.background",
    "foreground": "button.{intent}.foreground",
    "radius": "button.radius",
    "height": "button.{size}.height",
    "horizontalPadding": "button.{size}.paddingX"
  },
  "pencil": {
    "componentRefPattern": "ds.button.{intent}.{size}.{state}",
    "descendantOverrides": {
      "label.content": "props.label",
      "label.fill": "tokens.foreground",

```

```

    "root.fill": "tokens.background"
  }
},
"html": {
  "component": "Button",
  "props": {
    "intent": "intent",
    "size": "size",
    "children": "label"
  }
},
"hardRules": [
  "Must use a registered Button component/ref.",
  "Must not draw a raw rectangle to imitate a button.",
  "Label length must be <= 28 characters.",
  "Primary buttons: max one per action group.",
  "All colors must come from tokens."
]
}

```

这个文件就是你说的“AI 可以理解的完整映射”，但它更进一步：它既给 AI 读，也给 validator 读，也能编译到 Pencil 和 HTML。

9. 最小 tokens.json 示例

</> JSON



```

{
  "color.brand.primary": {
    "type": "color",
    "light": "#6750A4",
    "dark": "#D0BCFF"
  },
  "color.surface.default": {
    "type": "color",
    "light": "#FFFBFE",
    "dark": "#141218"
  },
  "color.text.default": {
    "type": "color",
    "light": "#1D1B20",
    "dark": "#E6E0E9"
  },
  "button.primary.background": {
    "type": "alias",
    "value": "color.brand.primary"
  },
  "button.primary.foreground": {
    "type": "alias",

```

```
"value": "color.surface.default"
},
"button.radius": {
  "type": "number",
  "value": 20
},
"button.md.height": {
  "type": "number",
  "value": 40
},
"space.4": {
  "type": "number",
  "value": 16
},
"type.display.large.size": {
  "type": "number",
  "value": 57
}
}
```

Pencil variables 正好可以承接这种结构：Pencil 变量类似 CSS custom properties / design tokens，修改变量会更新所有使用处，并且可以做 light/dark 这类 theme。  Pencil +1

10. 第一轮应该跑两条路线做对比

你不需要先争论哪条路线更好，直接做 A/B。

路线 A：Direct HTML

```
AI Design Kit
  ↓
Codex / Claude Code
  ↓
React / HTML
  ↓
浏览器截图 + lint + 人工看
```

目标：看直接写页面会不会乱用 CSS、乱写颜色、绕开组件。

路线 B：Pencil-mediated

```
AI Design Kit
  ↓
Codex / Claude Code 调 Pencil MCP
  ↓
生成 .pen
  ↓
Pencil screenshot / snapshot_layout
```

↓
同一语义结构生成 HTML

目标：看中间 `.pen` 是否让设计更可控、更容易检查、更容易纠错。

Codex 本身适合这种 repo 工作流，因为官方说明 Codex CLI 可以在本地终端读取、修改、运行代码，并支持 MCP 作为外部工具和上下文接入；Pencil 官方文档也明确有 Codex CLI 连接 Pencil MCP 的流程。  OpenAI 开发者 +2

11. 评估指标不要太复杂

第一轮只看 8 个指标：

1. 是否使用了注册组件
2. 是否出现 raw imitation component
3. 是否使用 tokens，而不是硬编码颜色/字号/间距
4. 是否违反 slot / props / variant 规则
5. 是否支持 light / dark theme
6. 是否出现 layout overlap / overflow
7. 人工需要改几次
8. HTML 是否仍然遵守组件契约

可以做一个简单 scorecard：

 Markdown 

```
# Scorecard
```

```
Task: SaaS landing page
```

```
## Direct HTML
```

- Component reuse: 6/10
- Token compliance: 4/10
- Layout stability: 7/10
- Manual corrections: 5
- Major violations:
 - created raw button styles
 - hardcoded brand color
 - pricing cards not using registered Card pattern

```
## Pencil-mediated
```

- Component reuse: 9/10
- Token compliance: 8/10
- Layout stability: 8/10
- Manual corrections: 2
- Major violations:
 - one card subtitle exceeded max length

这样你很快就知道 Pencil 中间层到底有没有价值。

12. Prompt 组 1：让 AI 编译组件库成 AI Design Kit

这个 prompt 用在 Codex / Claude Code 里。第一轮你可以给它 Material Web 或 Untitled React 的组件源码/文档路径，也可以让它先基于公开 docs 做 lite 版。

你现在要为一个 AI-controlled design POC 创建一个最小 AI Design Kit。

目标：

基于 Material Design 3 / Material Web 的设计原则，创建一个很小但严格的组件契约系统，用于后续

不要创建完整设计系统，只创建 POC 所需文件。

请创建以下文件：

```
design-system/tokens.json
design-system/token-map.json
design-system/usage-rules.json
design-system/retrieval-cards.json
design-system/pencil-library-spec.json
design-system/html-rendering-rules.json
```

```
design-system/components/button.contract.json
design-system/components/card.contract.json
design-system/components/hero.contract.json
design-system/components/feature-card.contract.json
design-system/components/pricing-card.contract.json
design-system/components/section.contract.json
design-system/components/footer.contract.json
```

要求：

1. 所有颜色、字号、间距、圆角都必须通过 tokens 表达。
2. 每个组件 contract 必须包含：
 - id
 - role
 - whenToUse
 - whenNotToUse
 - variants
 - props
 - slots
 - token bindings
 - pencil mapping
 - html mapping
 - hardRules
 - softRules
3. 不允许把组件说明写成长篇散文。必须是机器可读 JSON。
4. 每个 hardRule 以后都应该能被 validator 检查。
5. 第一轮只覆盖 SaaS landing page 场景，不要扩展到所有 Material 组件。

6. 输出后给我一个简短报告，说明哪些规则是硬约束，哪些只是风格建议。

完成定义：

- 所有 JSON 可解析。
- 每个 component id 在 retrieval-cards.json 里都有摘要。
- tokens.json 里至少包含 light/dark 两套关键 token。
- pencil-library-spec.json 说明每个组件应该如何变成 Pencil reusable/ref/descendants/sl

13. Prompt 组 2：让 AI 生成 Pencil library spec 或 .lib.pen

这一步先不追求完美的视觉组件库，先追求结构合法。

你现在要根据 design-system/pencil-library-spec.json 和 design-system/components/*.cor



目标：

生成 pencil/material-lite.lib.pen，包含最小 reusable components：

- ds.button.primary.md.default
- ds.button.secondary.md.default
- ds.card.default
- ds.hero.split
- ds.feature-card.default
- ds.pricing-card.default
- ds.section.default
- ds.footer.default

要求：

1. 必须使用 Pencil .pen JSON object tree。
2. 每个可复用组件必须设置 reusable: true。
3. 页面实例必须通过 type: "ref" 引用 reusable component。
4. 可编辑文本必须通过 descendants override 改写，不要复制整棵组件树。
5. 颜色、字体、间距、圆角必须使用 variables，例如 "\$color.brand.primary"，不要硬编码。
6. 支持 light/dark theme variables。
7. 容器组件必须使用 frame layout，而不是自由绝对定位。
8. Hero、Card、PricingCard 必须声明 slot 或可替换 children 的区域。
9. 每个组件 root 的 metadata 里必须包含：
 - componentId
 - contractVersion
 - allowedOverrides
 - tokenBindings

完成后：

- 检查 JSON 是否可解析。
- 列出每个 reusable component 的 id。
- 列出每个组件允许覆盖的 descendants path。

14. Prompt 组 3：让 AI 基于组件库生成 `.pen` 页面

这个 prompt 是最重要的，因为它测试“可控生成”。

你现在要基于已有 AI Design Kit 和 Pencil library 生成一个 SaaS landing page。



输入文件：

- design-system/tokens.json
- design-system/usage-rules.json
- design-system/retrieval-cards.json
- design-system/components/*.contract.json
- pencil/material-lite.lib.pen

输出：

- pencil/landing-page.pen
- pencil/landing-page-generation-report.md

任务：

设计一个 1440px 宽的 SaaS landing page，主题是：

"AI design system control plane for product teams"

页面结构：

1. Navbar
2. Hero
3. Feature grid
4. Pricing section
5. Footer

严格要求：

1. 只能使用 design-system/components 里注册过的组件。
2. 禁止用 raw rectangle/text/frame 模拟 Button、Card、PricingCard。
3. 所有颜色、字号、间距、圆角必须使用 tokens。
4. 所有按钮必须是 ds.button ref。
5. Primary CTA 在一个 action group 中最多只能出现一个。
6. Hero 的主标题不超过 80 字符。
7. Button label 不超过 28 字符。
8. Pricing section 必须有 3 个 pricing-card。
9. 不要创建未注册组件，除非只是布局用 section/frame，而且 metadata 标记为 layout-only。
10. 输出前检查是否违反 hardRules。

视觉方向：

- 高级、干净、B2B SaaS
- 使用 Material-like token structure，但不要做 Android 风格
- 充足留白
- 支持 light/dark theme

完成后：

1. 生成 .pen。
2. 在 report 里列出：
 - 使用了哪些组件
 - 使用了哪些 token

- 有无 `hardRule` 违规
- 有无 `raw imitation component`
- 有哪些地方你不确定

15. Prompt 组 4：让 AI 用 Pencil MCP 验证并修复

这一步是 Pencil 路线真正拉开差距的地方。

请使用 Pencil MCP 工具验证 `pencil/landing-page.peno`。

验证流程：

1. `get_editor_state({ include_schema: true })`
2. `batch_get` 读取页面主要节点和组件实例
3. `get_variables` 检查是否使用 `token variables`
4. `snapshot_layout` 检查布局结构、重叠、溢出
5. `get_screenshot` 渲染 `landing page` 截图
6. 如果发现问题，使用 `batch_design` 做最小修复
7. 修复后再次 `snapshot_layout` 和 `get_screenshot`
8. `save()`

硬性检查：

- 页面里不能出现看起来像按钮但不是 `ds.button ref` 的节点。
- 页面里不能出现看起来像卡片但不是 `ds.card / ds.pricing-card / ds.feature-card ref` 的节点。
- 除 `layout-only frame` 外，不能创建未注册视觉组件。
- 颜色不能硬编码为 `hex`，必须引用 `variables`。
- 组件实例应通过 `descendants` 覆盖文案，而不是复制组件内部结构。
- `layout` 不应有明显 `overlap`。
- `light/dark theme` 下文本需要保持可读。

输出：

- `pencil/landing-page-validation-report.md`
- 修复过的 `pencil/landing-page.pen`
- 导出的 `pencil/exports/landing-page.png`

Pencil 的 MCP 工具正好支持这些动作：`batch_get` 读层级，`get_variables/set_variables` 处理变量，`snapshot_layout` 分析布局，`get_screenshot` 渲染预览。  Pencil

16. Prompt 组 5：直接生成 HTML 的 baseline

这条路线拿来对比 Pencil 路线。

你现在要基于同一套 AI Design Kit 直接生成 HTML/React 页面。

输入文件：

```
- design-system/tokens.json
- design-system/usage-rules.json
- design-system/components/*.contract.json
- design-system/html-rendering-rules.json
```

输出：

```
- html/direct-html/index.html
- html/direct-html/styles.css
- html/direct-html/report.md
```

任务：

生成同一个 SaaS landing page：

"AI design system control plane for product teams"

严格要求：

1. 只能使用 `contracts` 里注册过的组件概念。
2. 不允许创建 `contract` 外的按钮、卡片、pricing card 样式。
3. CSS 中所有颜色、字号、间距、圆角必须来自 `tokens`。
4. 必须支持 `light/dark theme`。
5. `Button`、`Card`、`Hero`、`FeatureCard`、`PricingCard` 的 HTML 结构必须和 `html-rendering-rules` 一致。
6. 不要为了视觉效果写一次性 `class` 绕开组件规则。
7. 输出 `report`，说明是否有规则无法满足。

完成后：

- 启动本地预览。
- 检查页面是否可打开。
- 报告 `hardRules` 是否有违反。

这条线可以让你看到：没有 **Pencil** 中间层时，模型会不会更容易漂。

17. Prompt 组 6：主题变更测试

这个测试非常关键，因为它验证你最关心的 token 映射。

基于已有 `landing-page.pen` 和 `direct-html` 页面，做一次主题变更测试。

变更目标：

把整体品牌主色从当前颜色改成偏紫蓝的高级 B2B 色调，同时保持 `light/dark theme` 都可读。

规则：

1. 只能修改 `semantic token`，例如 `color.brand.primary`、`color.brand.secondary`、`focus.ring`。
2. 不允许直接修改组件内部 `fill/stroke`。
3. `Button`、`link`、`focus ring`、`selected state`、`hero accent` 应该通过 `token alias` 自动变化。
4. 不允许出现 `raw hex` 散落在组件节点或 CSS component class 中。
5. 修改后验证 `light/dark` 两种 `theme` 的文本可读性。

输出：

- 更新后的 `tokens.json`
- 更新后的 `pencil/landing-page.pen`

- 更新后的 `html/direct-html`
- `theme-change-report.md`

报告里必须说明：

- 修改了哪些 `token`
- 哪些组件受影响
- 有没有直接改组件颜色
- 有没有违反 `token alias` 规则

如果这一步跑通，说明你的“整体主体色调变化 → 组件对应部分自动变化”的想法是成立的。

18. Prompt 组 7：反向纠错记忆

你之前说“随用随建、建了存起来，也可以被纠正”，这一步可以直接验证。

下面是人工审核反馈，请不要只在对话里记住它，而是把它写入设计系统文件。



反馈：

在 Pricing Section 中，推荐套餐可以使用 `primary button`；其他套餐只能使用 `secondary button`。
`PricingCard` 内不能出现两个 `primary CTA`。

价格卡片的价格文本必须使用 `type.display.small`，不允许使用 `hero display token`。
`FeatureCard` 里的 `icon` 只能使用 `muted accent`，不允许使用 `primary brand fill`。

请修改：

- `design-system/components/pricing-card.contract.json`
- `design-system/components/feature-card.contract.json`
- `design-system/usage-rules.json`
- `design-system/retrieval-cards.json`

要求：

1. 把反馈变成 `hardRules` 或 `softRules`。
2. 如果能被机器检查，就写成 `hardRule`。
3. 更新后重新检查 `landing-page.pen` 和 `HTML` 是否违反这些规则。
4. 输出 `correction-report.md`，说明哪些反馈已经固化为规则。

这一步证明你的系统不是靠“聊天记忆”，而是靠文件化规则持续变强。

19. 最小验证任务集

我建议第一轮只跑 3 个任务。

Task 1:

生成 SaaS landing page



Task 2:

切换 brand primary color + dark mode

Task 3:

加入人工纠错规则后重新生成 pricing section

每个任务分别跑：

- A. Direct HTML
- B. Pencil-mediated

最后对比 scorecard。

20. 判断标准：什么结果说明这条路值得继续？

只要出现下面几个现象，就说明 Pencil 中间层值得继续：

1. Pencil 路线更少出现假组件
2. Pencil 路线更容易发现 layout / overlap / token 问题
3. Pencil 路线的人类修改次数更少
4. 主题变更更稳定
5. 纠错能沉淀到 contract，下一次明显减少同类错误

但如果结果是：

Direct HTML 也能稳定用组件
Direct HTML 的 token 遵守率很高
Pencil 生成反而增加复杂度
Pencil → HTML 损失语义

那你就应该把 Pencil 定位成 可选视觉评审层，而不是主链路。

21. 我对路线的最终建议

第一轮不要做：

Figma MCP → 抽完整组件库 → 自动生成完整 Pencil library → 自动生成设计系统 → 自动生成 HTML

这个太重，变量太多，失败了也不知道是哪一步的问题。

第一轮应该做：

已有公开组件体系
↓
AI 生成小型 Component Contracts

↓
AI 生成 Pencil library
↓
AI 基于 library 生成 .pen
↓
Pencil MCP 验证
↓
同一 contracts 生成 HTML
↓
和 Direct HTML 对比

一句话：

先证明“组件契约能约束 AI 设计”，再接 Figma；先证明 Pencil 中间层能降低违规率，再谈 Pencil→HTML 自动化。

我会把 POC 的核心假设写成这样：

Hypothesis:
Given a small component contract package,
an AI agent will produce more design-system-compliant UI when it must first create
compared with generating HTML directly.



这就是最短、最干净、最能验证其实这种简单 POC 不是我们的需求。我们既然要做这个事情，肯定是要做生产环境的，基于已有的产品组件库去做的事情。

做出来的东西就是有确定性的。因为如果你是直接生成 HTML 的话，其实有很多细节还要去调，对不上设计规范就很麻烦，而且这种细节很繁琐。但是你如果有组件库的话，这些细节几乎可以全部忽略掉。

只是在我们 AI design kit，我觉得你这个词特别好，就是 AI design kit。我们把这个组件库、设计规范翻译成了一个 AI

展开 ∨

你这个方向已经不是验证能否生成页面，而是做生产级 AI Design Kit 编译与执行系统；重点是把组件库变成可操作、可校验、结果确定的设计操作系统。

我会把“像人一样理解组件”拆成语义、场景、禁忌、组合语法和可执行操作，并把这些判断文件化、可检索、可测试。这样模型负责选择，系统负责约束和审计。

已思考 4m 35s >

你说得很准。**如果目标是生产环境，核心就不是 POC，也不是“把 Figma 组件翻译成 Pencil 组件”这么浅。**你真正要做的是把已有产品组件库编译成一套 **AI Design Kit**：它不

只是组件清单，而是一个让 AI 能够“像设计师一样使用组件”的操作系统。

我会这样定义：

AI Design Kit = 组件库 + 设计规范 + 使用语义 + 组合模式 + token 依赖 + 约束规则 + 反例 + 编译映射 + 校验器 + 纠错记忆。

它的目标不是“模型看懂文档”，而是让模型在设计时具备类似人类设计师的判断：什么时候用哪个组件、为什么用、怎么组合、什么情况下不能用、哪些属性联动、哪些视觉细节不该手调、出了错怎么修。

1. 你这个方向是对的，但要把“理解”拆开

你说“彻底像人一样理解组件怎么用”，我觉得要拆成 8 种理解。

第一是 **语义理解**：这个组件到底代表什么。比如 `Banner` 不是一个带图的卡片，而是“在页面关键位置表达提醒、推广或状态变化的高可见度信息区域”。

第二是 **场景理解**：什么时候该用，什么时候不该用。比如 `Toast` 适合短暂反馈，不适合承载需要用户仔细阅读的错误修复说明；`Modal` 适合阻断式决策，不适合普通信息展示。

第三是 **组理解**：它能和谁放在一起，谁只能放在它的 slot 里，谁不能嵌套谁。比如 `PricingCard` 可以放进 `PricingGrid`，但不应该随便塞进 `FeatureGrid`。

第四是 **层级理解**：这个组件在信息架构里的角色。比如 `Hero` 是页面主叙事入口，`FeatureCard` 是辅助论点，`CTASection` 是转化收口。

第五是 **状态理解**：default、hover、selected、disabled、error、loading、empty、success 这些状态之间怎么切，哪些状态必须带文案或辅助说明。

第六是 **token 理解**：整体主色、暗色亮色、字体比例、圆角、密度变化时，组件内部哪些部分应该跟着变，哪些不应该变。

第七是 **审美和规范理解**：不是“能画出来”，而是符合这个产品自己的气质。比如同样是 SaaS，Linear、Stripe、Notion、Vercel 的组件使用节奏完全不同。

第八是 **反例理解**：人类设计师很大一部分能力来自知道“这样用很怪”。AI Design Kit 里必须存 bad examples 和 forbidden patterns，而不只是 good examples。

所以，AI Design Kit 不能只是：

Button 有 primary / secondary / ghost
Card 有 title / body / footer



它应该是：

这个组件是什么
解决什么问题
适合什么场景
不适合什么场景
有哪些变体
每个变体的语义差异是什么
怎么和其他组件组合
哪些属性联动 token
哪些地方允许改
哪些地方永远不能改
常见错误是什么
错误后怎么自动修复



2. 生产环境里，AI Design Kit 应该是“编译出来的专家系统”

你说这些层不能由人手写，这点我同意。生产路径应该是：

已有产品组件库

- └ Figma library
- └ Code components
- └ Code Connect
- └ Storybook / docs
- └ 已有产品页面
- └ 设计规范文档
- └ token 文件
- └ 历史设计稿

↓ AI + MCP + 静态分析 + 视觉分析

AI Design Kit Compiler

↓

可执行 AI Design Kit

- └ component contracts
- └ token graph
- └ composition rules
- └ usage ontology
- └ Pencil library
- └ semantic ops schema
- └ validators
- └ retrieval index
- └ examples / anti-examples
- └ correction memory



Figma MCP 和 Code Connect 在这条链路里很关键，但它们只是输入源。Figma MCP 能获取设计上下文、变量、库组件、截图、Code Connect 映射等信息；Code Connect 还能把 Figma 组件的当前属性、import 语句、组件使用代码、prop mapping、自定义使用说明放进上下文。  Figma 开发者文档 +2

Pencil 则适合作为可执行画布。Pencil 的 `.pen` 是 JSON object tree，支持 frame/text/rectangle 等图形对象、flexbox-style layout、reusable component、`ref` instance、`descendants` `override`、slot、variables/themes。  Pencil +3

所以生产架构里最好不要让 AI 直接写 HTML，也不要让 AI 直接乱写 Pencil raw nodes。正确结构是：

```
AI 先操作 AI Design Kit 的语义层
↓
语义操作经过规则校验
↓
再编译成 Pencil .pen
↓
Pencil 截图 / 布局 / token / 组件实例验证
↓
同一套语义操作也可以编译成 HTML / React
```

3. “确定性”来自哪里？

你说“做出来的东西就是有确定性的”，这个非常重要。生产级系统里，确定性不应该依赖模型稳定发挥，而应该来自这几个机制：

1. 组件只能从 `registry` 里选
2. 属性只能从 `contract` 里改
3. slot 只能接受声明过的组件
4. token 只能走 `token graph`
5. layout 只能用 `layout recipe`
6. 视觉输出必须经过 `Pencil screenshot / snapshot_layout`
7. 代码输出必须经过 `component API mapping`
8. 所有违规必须被 `validator` 拦截

模型负责的是：

```
理解需求
选择合适组件
决定页面结构
生成文案
选择合法变体
```

提出合法操作
解释设计意图

系统负责的是：

检查是否合法
编译成 Pencil
渲染成效果图
验证布局
验证 token
验证组件身份
生成 HTML / React
拒绝违规输出



这就像人类设计师使用 Figma 组件库：设计师可以做判断，但很多细节由组件库兜底。你要把这个机制 AI 化。

4. AI Design Kit 的核心文件结构

生产级 AI Design Kit 我建议长这样：

```
ai-design-kit/  
  kit.json  
  
  ontology/  
    component-roles.json  
    page-patterns.json  
    product-scenarios.json  
    information-architecture.json  
  
  tokens/  
    tokens.json  
    token-alias-graph.json  
    theme-modes.json  
    token-usage-rules.json  
  
  components/  
    button.contract.json  
    card.contract.json  
    modal.contract.json  
    data-table.contract.json  
    command-menu.contract.json  
    empty-state.contract.json  
    pricing-card.contract.json  
  
  compositions/  
    form-patterns.json  
    dashboard-patterns.json
```



```
settings-page-patterns.json
onboarding-patterns.json
pricing-page-patterns.json

rules/
  global-hard-rules.json
  global-soft-rules.json
  accessibility-rules.json
  responsive-rules.json
  brand-rules.json
  anti-patterns.json

mappings/
  figma-to-kit.json
  kit-to-pencil.json
  kit-to-react.json
  kit-to-css-vars.json
  code-connect-map.json

pencil/
  product.lib.pen
  fixtures.pen
  example-pages.pen

examples/
  good/
  bad/
  edge-cases/
  canonical-screens/

retrieval/
  component-cards.json
  scenario-cards.json
  pattern-cards.json
  embeddings.index

validators/
  component.schema.json
  operation.schema.json
  token.schema.json
  rule-tests.json

memory/
  corrections.jsonl
  decisions.jsonl
  rejected-generations.jsonl
```

这个东西才叫 AI Design Kit。它不是“给模型看的文档”，而是一个同时服务于：

检索
推理



生成
编译
验证
纠错
回放
版本管理

的生产系统。

5. 最重要的不是组件 contract，而是 ontology

如果你希望 AI “像人一样理解”，那单个组件 contract 还不够。真正要补的是 设计系统语义本体 ontology。

比如你不能只告诉它：

```
Button 有 primary、secondary、ghost
```



你要告诉它：

```
primary action = 当前上下文中最希望用户采取的动作  
secondary action = 有用但不是主路径的动作  
destructive action = 会造成不可逆或高风险变化的动作  
navigation action ≠ submit action  
inline action ≠ page-level CTA
```



这应该写成机器可读的 ontology：

JSON



```
{  
  "concepts": {  
    "primary_action": {  
      "definition": "The most important action in a local decision context.",  
      "allowedComponents": ["ds.button", "ds.menuItem"],  
      "preferredVariants": {  
        "ds.button": "primary"  
      },  
    },  
    "rules": [  
      "max_one_per_action_group",  
      "must_have_clear_action_label",  
      "must_not_be_used_for_destructive_action_unless_flow_is_destructive"  
    ],  
  },  
  "secondary_action": {  
    "definition": "An action that supports the main path but should not visually compet  
    "preferredVariants": {
```

```

      "ds.button": "secondary"
    }
  },
  "destructive_action": {
    "definition": "An action that deletes, cancels, removes, revokes, or permanently ch
    "preferredComponents": ["ds.button", "ds.confirmationModal"],
    "rules": [
      "requires_confirmation_if_high_risk",
      "must_use_destructive_variant",
      "must_not_be_default_primary_in_non_destructive_context"
    ]
  }
}
}
}

```

这层会让 AI 从“知道 Button 属性”升级到“知道为什么用这个 Button”。

6. Component Contract 应该长什么样

生产环境的组件 contract 不只是 props 表。它应该包括 语义、结构、视觉、token、行为、代码、Pencil、规则、例子、反例。

以 `Button` 为例：

</> JSON



```

{
  "id": "ds.button",
  "version": "3.4.2",
  "displayName": "Button",
  "semanticRole": "action_trigger",

  "meaning": {
    "definition": "A button triggers an action in the current context.",
    "useWhen": [
      "The user needs to submit, confirm, continue, save, create, or trigger an operation
      "The action is explicit and should be visually discoverable."
    ],
    "doNotUseWhen": [
      "The user is simply navigating to related documentation; use ds.link instead.",
      "The whole card is clickable; do not place redundant buttons unless needed.",
      "The action is repeated in a dense list; use ds.iconButton or ds.menuItem."
    ]
  },

  "variants": {
    "intent": {
      "values": ["primary", "secondary", "ghost", "destructive"],
      "semantics": {

```

```

    "primary": "The main action in a local action group.",
    "secondary": "A supporting action.",
    "ghost": "Low-emphasis action.",
    "destructive": "Dangerous or irreversible action."
  }
},
"size": {
  "values": ["sm", "md", "lg"],
  "rules": {
    "sm": "Dense interfaces, tables, compact toolbars.",
    "md": "Default product UI.",
    "lg": "Marketing or page-level CTA."
  }
}
},
"props": {
  "label": {
    "type": "string",
    "required": true,
    "maxLength": 28,
    "contentRules": [
      "Use a verb-first action label.",
      "Avoid vague labels such as Submit unless form context is obvious."
    ]
  },
  "leadingIcon": {
    "type": "componentRef",
    "accepts": ["ds.icon"],
    "required": false
  },
  "ariaLabel": {
    "type": "string",
    "requiredWhen": "label is visually hidden or iconOnly is true"
  }
},
"compositionRules": [
  {
    "id": "primary_max_one_per_action_group",
    "severity": "hard",
    "condition": "parent.role == action_group",
    "assert": "count(children where component == ds.button and variant.intent == primary) <= 1"
  },
  {
    "id": "destructive_requires_context",
    "severity": "hard",
    "condition": "variant.intent == destructive",
    "assert": "ancestor.intent in ['delete_flow', 'remove_flow', 'danger_zone']"
  }
]

```

```

"tokenBindings": {
  "root.fill": "button.{intent}.background",
  "root.cornerRadius": "button.radius",
  "root.height": "button.{size}.height",
  "label.fill": "button.{intent}.foreground",
  "label.font": "type.label.{size}"
},

"pencil": {
  "componentRefPattern": "ds.button.{intent}.{size}.default",
  "allowedDescendantOverrides": {
    "label.content": "props.label",
    "icon-left.children": "props.leadingIcon"
  },
  "forbiddenOverrides": [
    "root.fill",
    "root.cornerRadius",
    "label.fontSize"
  ]
},

"code": {
  "framework": "react",
  "import": "@components/ui/Button",
  "component": "Button",
  "propMap": {
    "intent": "variant",
    "size": "size",
    "label": "children",
    "leadingIcon": "startIcon",
    "ariaLabel": "aria-label"
  }
},

"examples": {
  "good": [
    "Save changes",
    "Create workspace",
    "Invite member"
  ],
  "bad": [
    "Click here",
    "Submit",
    "Yes"
  ]
}
}

```

这已经不是低层翻译了。它让 AI 知道：

Button 是什么
什么时候用
什么时候不用
哪个 variant 代表什么
文案怎么写
和上下文有什么关系
token 怎么绑定
Pencil 怎么生成
代码怎么生成
哪些事不能做



7. AI Design Kit Compiler 应该怎么自动生成这些东西

你说不希望人手工做，这完全合理。生产环境可以用一个多阶段 compiler 自动生成。

第一阶段：采集

从这些地方拿信息：

Figma library
Figma component sets
Figma variables
Figma published components
Code Connect
组件源码
Storybook
设计系统文档
真实产品页面
历史设计稿
现有 CSS / token 文件



Figma MCP 里 `get_variable_defs` 可以返回选中内容使用的颜色、间距、字体等变量；
`search_design_system` 可以搜索设计库中的组件、变量和样式；`get_code_connect_map`
可以拿 Figma instance 和代码组件的映射；`get_context_for_code_connect` 能返回组件
属性、variants、descendant tree 和 property references。 [Figma 开发者文档 +1](#)

第二阶段：归一化

把 Figma / code / docs 的不同名字统一：

Figma: Button / Type=Primary / Size=Large
Code: `<Button variant="primary" size="lg" />`
Pencil: `ds.button.primary.lg.default`
AI Kit: `ds.button { intent: primary, size: lg }`



这一步会生成：

```
canonical component id
canonical prop name
canonical variant name
canonical token name
canonical slot name
```



第三阶段：语义推断

AI 从 docs、Code Connect instructions、Storybook examples、真实页面使用方式里推断：

```
这个组件解决什么问题
常见使用场景
常见组合方式
哪些变体在什么上下文用
哪些属性是内容属性
哪些属性是结构属性
哪些属性是视觉属性
哪些属性不应该被 AI 改
```



这里应该用多 agent，不要一个 prompt 生成完：

```
Extractor agent: 提取事实
Semantic agent: 总结使用语义
Rule agent: 转成 hard / soft rules
Mapping agent: 生成 Figma/Pencil/Code 映射
Critic agent: 找矛盾和缺漏
Validator agent: 生成测试用例
```



第四阶段：生成 contract draft

AI 输出结构化 contract。每条规则都要带来源和置信度：

```
<> JSON

{
  "rule": "primary_max_one_per_action_group",
  "source": [
    "storybook/Button.stories.tsx",
    "figma examples: Pricing page, Settings page",
    "design-docs/actions.md"
  ],
  "confidence": 0.92,
  "needsHumanReview": false
}
```



低置信度才需要人看。

第五阶段：反向验证

这一步非常关键：不要只相信 AI 生成的 contract。

你要用 contract 去重建已有页面，然后比较：

已有 Figma 设计稿

vs

AI Design Kit 重建出来的 Pencil 设计图

检查：

组件是否识别正确

variant 是否对

token 是否对

slot 是否对

结构是否对

视觉截图是否接近

Pencil MCP 有 `get_screenshot`、`snapshot_layout`、`batch_get`、`get_variables/set_variables`，正好适合这个验证闭环。 

第六阶段：生成 Pencil library

把 AI Design Kit 编译成：

`product.lib.pen`

里面的组件必须是 Pencil reusable component，实例必须用 `ref`，文本和局部内容通过 `descendants` override，容器区域用 slot，颜色/字体/间距用 variables。Pencil 文档里明确支持 reusable → `ref` instance、`descendants` override、children replacement、slot 和 variables/themes，这些能力正好对应组件库生产化。 

8. 生产生成流程应该是这样的

用户说：

帮我做一个企业后台的用户权限管理页面。

系统不应该直接让 AI 画页面。它应该走这个流程：

1. Intent Understanding

页面类型：enterprise admin page

业务对象：user permission

信息结构: table + filters + role management + bulk actions

风险动作: remove access / change role

2. Retrieve AI Design Kit

加载相关 ontology:

- admin_page
- data_management
- permission_flow
- destructive_action

加载相关 components:

- ds.pageShell
- ds.topBar
- ds.filterBar
- ds.dataTable
- ds.badge
- ds.button
- ds.dropdownMenu
- ds.confirmationModal
- ds.emptyState

加载相关 patterns:

- table_with_filters
- bulk_action_bar
- destructive_confirmation

3. Generate semantic design plan

不是 .pen, 不是 HTML, 而是:

页面由哪些组件组成

每个组件承担什么语义角色

每个 slot 填什么

每个 action 的风险等级

每个 variant 为什么选

4. Convert to operations

- component.insert
- component.set_prop
- component.fill_slot
- component.bind_data
- component.set_state
- token.reference

5. Validate

组件合法

slot 合法

variant 合法

token 合法

responsive 合法

accessibility 合法

destructive action 合法

6. Compile to Pencil

用 `product.lib.pen` 里的 `reusable refs`
用 `descendants` 写文案
用 `slot` 组合
用 `variables` 控制视觉

7. Visual QA

`snapshot_layout`
`get_screenshot`
检查 `overlap` / `overflow` / `hierarchy` / `density`

8. Compile to code

同一套语义 `graph` 编译成 `React` / `HTML`
使用真实组件 `API`

这才是生产级闭环。

9. 让 AI “像人一样理解”的关键：Examples + Anti-examples + Critic

人类设计师不是只读组件文档学会的，而是看大量例子、被 review、知道什么不对。AI Design Kit 里也必须有这三类东西。

Good examples

JSON



```
{
  "component": "ds.emptyState",
  "scenario": "No search results in user table",
  "whyGood": [
    "Uses empty state instead of error alert",
    "Has clear title, explanation, and reset-filter action",
    "Primary action is not over-emphasized"
  ],
  "structure": {
    "title": "No users found",
    "description": "Try adjusting your filters or search terms.",
    "action": {
      "component": "ds.button",
      "variant": "secondary",
      "label": "Clear filters"
    }
  }
}
```

Bad examples

</> JSON



```
{
  "component": "ds.alert",
  "scenario": "No search results in user table",
  "whyBad": [
    "No results is not an error",
    "Alert creates unnecessary urgency",
    "Primary destructive color is misleading"
  ],
  "repair": {
    "replaceWith": "ds.emptyState"
  }
}
```

Critic rules

</> JSON



```
{
  "critic": "admin_page_design_reviewer",
  "checks": [
    "Does the page use table patterns for dense management tasks?",
    "Are destructive actions visually separated and confirmed?",
    "Are filters grouped before the table?",
    "Are empty/loading/error states represented?",
    "Is the primary action unique and contextually correct?"
  ]
}
```

生成之后，不是只看 schema。还要让 critic 根据 AI Design Kit 的 ontology 和 examples 做设计 review。

10. 上下文窗口怎么解决

生产环境不能把整个组件库塞进上下文。要做分层上下文。

我建议每个组件有四种上下文形态：

Level 0: index

一句话摘要，用于搜索和初筛。

Level 1: card

100~300 token，说明用途、主要 variants、主要禁忌。

Level 2: contract

完整 JSON，用于真正使用该组件时加载。



Level 3: deep context

示例、反例、截图、源码、Figma 节点、历史纠错，只在复杂场景加载。

一次生成页面时，模型看到的应该是：

全局设计原则摘要
相关 ontology 摘要
候选组件 card
实际使用组件的 full contract
相关 composition pattern
必要 token 子图
少量 good / bad examples



也就是说，模型不是“记住整个组件库”，而是通过检索拿到当前任务所需的最小知识包。

11. 你要防止 AI Design Kit 变成“低层翻译”

低层翻译是：

Figma Button → Pencil Button
Figma Card → Pencil Card
Figma Variables → Pencil Variables



高级 AI Design Kit 是：

Action ontology
→ Button / Link / MenuItem / IconButton 选择逻辑

Content density ontology
→ Table / Card / List / Timeline 选择逻辑

Risk ontology
→ destructive variant / confirmation modal / danger zone 选择逻辑

Page pattern ontology
→ settings / dashboard / onboarding / checkout / admin page 结构逻辑

Token dependency graph
→ 品牌、主题、密度、字体、暗色亮色联动逻辑

Composition grammar
→ 哪些组件可以组合，怎么组合，什么顺序，什么层级



所以你最终不是在做：

Figma to Pencil converter



而是在做：

Product Design System → AI-operable Design Intelligence



Figma 和 Pencil只是输入/执行层。

12. 我建议的产品级模块

生产产品可以拆成 6 个核心模块。

1. AI Design Kit Builder

负责从 Figma、代码、文档、Storybook、已有页面自动生成 ADK。

输出：

```
component contracts
token graph
ontology
composition patterns
Pencil library
validators
retrieval index
```



2. AI Design Kit Reviewer

不是让人从 0 写规则，而是让人 review AI 自动生成的高价值差异：

```
低置信度规则
规则冲突
组件语义不清
高频组件
高风险组件
生成失败案例
```



人类只处理例外和纠错。

3. Semantic Design Runtime

用户自然语言请求进入后，先生成 semantic plan 和 semantic ops，不直接生成视觉节点。

4. Pencil Compiler

把 semantic ops 编译成 Pencil：

```
component.insert → ref instance
setProp(text) → descendants override
fillSlot → slot children replacement
token.update → variables
theme.switch → theme mode
```



5. Design Validator

校验：

```
component identity
props
variants
slots
tokens
layout
accessibility
responsive
brand
anti-patterns
```



6. Code Compiler

同一套 semantic ops 编译成真实代码组件，而不是从 Pencil 视觉图硬转 HTML。

13. 很关键的一点：Pencil 和 HTML 都应该是“编译目标”

生产级路线不要做：

```
Pencil → HTML
```



这会丢语义。更稳的是：

```
Semantic Design Graph
├─ Pencil .pen
└─ React / HTML / Vue / SwiftUI
```



Pencil 的价值是：

```
设计图
可视化
AI 可操作画布
组件实例验证
截图 QA
设计 review
```



代码的价值是：

真实组件 API
可运行页面
业务接入
工程交付



它们应该来自同一个 AI Design Kit 和同一个 semantic graph，而不是互相反推。

14. 生产环境的“确定性生成”应该长这样

你可以把每次生成固定成一个 pipeline：

```
User Request
↓
Intent Object
↓
Relevant Kit Retrieval
↓
Semantic Plan
↓
Component Operation List
↓
Static Validation
↓
Pencil Compilation
↓
Pencil MCP Verification
↓
Visual Critic
↓
Repair Loop
↓
Final .pen + screenshot + code
```



每一步都有结构化产物：

```
intent.json
retrieved-context.json
plan.json
ops.json
validation-report.json
design.pen
screenshot.png
visual-review.json
repair-log.json
generated-code/
```



这会让整个过程可审计、可回滚、可复现。

15. 你的“AI Design Kit”最好分成三层成熟度

Level 1：组件可用

AI 知道有哪些组件、props、variants、tokens。

这只能保证“不乱画”。

Level 2：组件可控

AI 只能通过合法 operation 使用组件，违规会被 validator 拦截。

这能保证“不违背硬规范”。

Level 3：组件有设计判断

AI 知道场景、意图、信息层级、组合模式、反例、审美偏好。

这才接近你说的“像人一样理解”。

你现在真正想做的是 Level 3，不是 Level 1。

16. 一句非常重要的产品原则

你不应该试图把所有“理解”都塞给模型。你应该把理解拆成：

模型负责语义推理

Kit 负责长期知识

Validator 负责硬约束

Compiler 负责确定性输出

Critic 负责设计审查

Memory 负责持续学习



这比“让模型百分百理解组件”更可靠。

真正的生产级目标应该是：

AI 设计师可以提出方案，但 AI Design Kit 决定它能不能落地。

17. 最终结论

你现在的判断是对的，而且方向应该更坚定一点：

不要把 AI Design Kit 做成简单的组件翻译层。

它应该是产品设计系统的 AI 运行时。

它应该知道：

```
组件是什么
组件为什么存在
组件解决什么场景
组件不能解决什么场景
组件怎么组合
组件怎么响应主题变化
组件怎么映射到 Pencil
组件怎么映射到代码
组件怎么被验证
组件怎么被纠错
```



所以真正的生产路线是：

```
已有产品组件库
→ AI Design Kit Compiler
→ AI Design Kit Review
→ Semantic Design Runtime
→ Pencil Compiler
→ Pencil Visual QA
```



你可以直接调用 code x 吗？我想的是你先把这个东西，就是整套系统，帮我设计成一组 skills 的组合。

可以。我先把它整理成一组可由 Codex/Claude Code/Cursor 执行的 AI Design Kit Skills pipeline，并说明我不能直接调用你的 Codex 环境。每个 skill 都会按真实生产流程定义输入、输出、边界和可验证产物。你这个想法里最有价值的词就是 **AI Design Kit**。它不是设计系统的副本，而是设计系统面向 AI 的“认知层 + 操作层 + 约束层”。只有做到这一层，AI 生成的东西才会从“看起来还行”变成“稳定符合产品设计原则”。

已思考 4m 49s >

可以，但分两层说清楚：

我在这个 ChatGPT 对话里不能直接启动你的本地 Codex / CodeX 进程，也不能直接连你机器上的 Pencil、Figma MCP、repo。

但 Codex 本身现在确实支持 **Skills、MCP、AGENTS.md、CLI/IDE/App** 这一套组合：官方文档里写得很明确，Codex Skills 可用于 CLI、IDE extension 和 Codex app；Skill 是带 `SKILL.md` 的目录，可以包含 scripts、references、assets；Codex 会先只看 skill 的 name/description，需要时再加载完整 `SKILL.md`，这正好适合你说的“上下文不要塞太多”。 OpenAI 开发者 +1 Codex 也支持 MCP server 配置，官方例子里甚至包含 Figma MCP；它支持本地 stdio server、streamable HTTP server、OAuth/bearer token、project-scoped

`.codex/config.toml` 等。  OpenAI 开发者 另外，Codex 会读取 `AGENTS.md` 作为项目级长期指令，且可以从 repo root 到当前目录逐层合并，这适合放全局生产规范。  OpenAI 开发者

所以正确做法是：

```
AGENTS.md          # 项目最高层长期约束
.agents/skills/*    # 一组专门的 AI Design Kit skills
.codex/config.toml  # MCP 配置: Figma / Pencil / GitHub / Browser / local validation
scripts/*          # 确定性校验、编译、diff、schema 检查
```

我建议你把整套系统设计成 **12 个 Skills + 1 个 AGENTS.md + 1 套 MCP 配置 + 一组确定性 scripts**。

1. 总体结构：不要做一个巨大的 Skill

这个系统不能是一个 `$ai-design-kit` 万能 Skill。那样会变成大 prompt，最后还是不稳定。

应该是：

```
adk-orchestrator
├─ adk-source-ingestion
├─ adk-component-semantics
├─ adk-token-graph
├─ adk-composition-grammar
├─ adk-contract-author
├─ adk-contract-reviewer
├─ adk-pencil-library-compiler
├─ adk-pencil-page-designer
├─ adk-pencil-validator
├─ adk-code-compiler
├─ adk-feedback-learner
└─ adk-release-manager
```

一句话：

Orchestrator 负责流程调度；每个专业 Skill 只做一件事；真正的确定性靠 scripts 和 validator。

2. 推荐仓库结构

你可以直接在产品 repo 或单独的 `ai-design-kit` repo 里放：



```
repo/
  AGENTS.md

  .codex/
    config.toml

  .agents/
    skills/
      adk-orchestrator/
        SKILL.md
        references/
          pipeline.md

      adk-source-ingestion/
        SKILL.md
        references/
          figma-mcp-playbook.md
          codebase-scan-playbook.md
        scripts/
          normalize_figma_snapshot.ts
          scan_storybook.ts

      adk-component-semantics/
        SKILL.md
        references/
          semantic-ontology-guide.md
          component-role-taxonomy.md

      adk-token-graph/
        SKILL.md
        scripts/
          build_token_graph.ts
          detect_token_alias_cycles.ts

      adk-composition-grammar/
        SKILL.md
        references/
          page-pattern-taxonomy.md
          slot-grammar-guide.md

      adk-contract-author/
        SKILL.md
        references/
          component-contract-schema.md
        scripts/
          validate_contracts.ts

      adk-contract-reviewer/
        SKILL.md
        references/
          contradiction-checklist.md
```

```
anti-pattern-checklist.md

adk-pencil-library-compiler/
  SKILL.md
  references/
    pencil-mapping-guide.md
  scripts/
    compile_pencil_library.ts
    validate_pen.ts

adk-pencil-page-designer/
  SKILL.md
  references/
    semantic-ops-guide.md
    page-generation-policy.md

adk-pencil-validator/
  SKILL.md
  scripts/
    validate_pen_component_usage.ts
    validate_pen_tokens.ts
    compare_layout_snapshot.ts

adk-code-compiler/
  SKILL.md
  references/
    react-binding-guide.md
  scripts/
    compile_react_from_semantic_graph.ts
    validate_component_api_usage.ts

adk-feedback-learner/
  SKILL.md
  scripts/
    apply_correction_to_contract.ts
    append_bad_example.ts

adk-release-manager/
  SKILL.md
  scripts/
    build_kit_manifest.ts
    diff_kit_versions.ts
```

同时有一套产物目录：

```
ai-design-kit/
  kit.json

  sources/
    figma.snapshot.json
    code.snapshot.json
```



```
storybook.snapshot.json
docs.snapshot.json

ontology/
  component-roles.json
  page-patterns.json
  intent-taxonomy.json

tokens/
  tokens.json
  token-graph.json
  theme-modes.json
  token-bindings.json

components/
  button.contract.json
  modal.contract.json
  data-table.contract.json
  ...

compositions/
  dashboard-page.pattern.json
  settings-page.pattern.json
  pricing-page.pattern.json
  form-flow.pattern.json

mappings/
  figma-to-kit.json
  kit-to-pencil.json
  kit-to-react.json

pencil/
  product.lib.pen
  examples.pen
  generated/
    user-permissions-page.pen

semantic/
  generated/
    user-permissions-page.graph.json
    user-permissions-page.ops.json

reports/
  validation/
  visual-review/
  release/
```

3. AGENTS.md：最高层生产规则

这个文件不要太长。它不是组件知识库，而是强制 Codex 每次都按你的生产流程走。

建议内容：

</> Markdown



```
# AGENTS.md
```

```
## Product goal
```

```
This repository builds and uses an AI Design Kit: a machine-operable representation of our
```

```
The agent must never treat design generation as freeform HTML/CSS or freeform canvas draw
```

```
## Core rule
```

```
All generated design work must flow through:
```

1. AI Design Kit retrieval
2. semantic plan
3. semantic component operations
4. contract validation
5. Pencil compilation
6. Pencil validation
7. optional code compilation

```
## Never do this
```

- Do not invent unregistered UI components.
- Do not draw raw rectangles/text to imitate Button, Card, Modal, Table, Badge, Input, or
- Do not hardcode colors, font sizes, radii, shadows, or spacing when a token exists.
- Do not bypass component contracts.
- Do not compile code directly from visual nodes when a semantic graph exists.
- Do not update component rules only in prose; write them into contracts, rules, examples

```
## Required design artifacts
```

```
For every generated screen, produce:
```

- semantic/*.graph.json
- semantic/*.ops.json
- pencil/generated/*.pen
- reports/validation/*.json
- reports/visual-review/*.md

```
## Preferred route
```

```
Use `sadm-orchestrator` for all end-to-end AI design tasks.
```

```
Use specialist skills only when working on one stage.
```

这个文件负责“世界观”和“底线”。具体流程放 Skills。

4. Skill 1: `adk-orchestrator`

定位：总调度。

触发时机：

用户说：

- 生成一个页面
- 基于组件库做设计
- 从 Figma / 组件库生成 AI Design Kit
- 编译 Pencil
- 校验设计是否符合组件库
- 从设计生成代码



`SKILL.md` 核心内容：

`</>` Markdown



```
---
name: adk-orchestrator
description: Orchestrates production AI Design Kit workflows: ingest existing design syst
---

# ADK Orchestrator

## Mission

Coordinate the AI Design Kit pipeline. Never perform specialist work directly if a specia

## Workflow selection

If the user asks to build or update the AI Design Kit, run:

1. adk-source-ingestion
2. adk-component-semantics
3. adk-token-graph
4. adk-composition-grammar
5. adk-contract-author
6. adk-contract-reviewer
7. adk-pencil-library-compiler
8. adk-release-manager

If the user asks to generate a screen or design:

1. retrieve relevant kit files
2. produce semantic plan
3. produce semantic ops
4. validate contracts
5. run adk-pencil-page-designer
```

6. run adk-pencil-validator
7. optionally run adk-code-compiler

If the user gives feedback or corrections:

1. classify correction
2. run adk-feedback-learner
3. rerun affected validators
4. update reports

Required outputs

Every end-to-end run must output:

- intent.json
- retrieved-context.json
- semantic plan
- semantic ops
- validation report
- Pencil output path
- unresolved issues

这个 Skill 不做细节，只保证流程不跑偏。

5. Skill 2: `adk-source-ingestion`

定位：从真实生产源头抽取素材。

输入：

```
Figma library
Figma MCP
Code Connect
组件源码
Storybook
设计规范文档
历史页面
tokens
```



输出：

```
sources/figma.snapshot.json
sources/code.snapshot.json
sources/storybook.snapshot.json
sources/docs.snapshot.json
sources/usage-examples.json
```



核心规则：



```
---
name: adk-source-ingestion
description: Ingests production design system sources into normalized snapshots: Figma co
---

# Source Ingestion

## Inputs

Prefer sources in this order:
1. Figma MCP / Figma API
2. Code Connect
3. component source code
4. Storybook stories
5. design docs
6. production screen examples

## Extraction requirements

For every component, extract:
- canonical name candidates
- Figma key / node id
- component set and variants
- component properties
- variables used
- nested instances
- slot-like areas
- screenshot or thumbnail references
- code component import path
- prop names and types
- Storybook examples
- docs usage notes
- real usage examples

## Output rules

Do not infer final semantics here. Only extract facts and source evidence.

Write all output to sources/*.snapshot.json.
```

这个 Skill 严禁“脑补”。它只采集事实。

6. Skill 3: `adk-component-semantics`

定位：让组件“有意义”。

它把采集来的事实转成 ontology。

输出:

```
ontology/component-roles.json
ontology/intent-taxonomy.json
ontology/component-decision-tree.json
```



核心内容:

</> Markdown



```
---
name: adk-component-semantics
description: Converts extracted component facts into design-system semantics: what each c
---

# Component Semantics

## Goal

Infer human-level design meaning from source evidence.

For each component, define:
- semantic role
- jobs-to-be-done
- whenToUse
- whenNotToUse
- variant semantics
- state semantics
- related components
- replacement logic
- user intent mapping
- risk level, if any

## Important distinction

Do not describe visuals only. Describe design meaning.

Bad:
"Button is a rounded rectangle with text."

Good:
"Button triggers an explicit user action in the current decision context."

## Evidence policy

Every semantic claim must include:
- sourceEvidence
- confidence
- needsReview boolean
```

Low-confidence claims must not become hard rules.

这个 Skill 是你说的“像人一样理解”的核心之一。

7. Skill 4: `adk-token-graph`

定位：处理主色、暗色亮色、字体、密度、状态联动。

输出：

```
tokens/tokens.json
tokens/token-graph.json
tokens/theme-modes.json
tokens/token-bindings.json
```



核心内容：

`</>` Markdown



```
---
name: adk-token-graph
description: Builds a token dependency graph from Figma variables, CSS variables, code to
---

# Token Graph Builder

## Token levels

Normalize tokens into:
1. primitive tokens
2. semantic tokens
3. component tokens
4. state tokens
5. Pencil variables
6. code variables

## Required graph

For every token, record:
- id
- type
- value or alias
- modes
- references
- consumers
- component bindings
- source evidence
```

```
## Theme behavior
```

```
Do not update component colors directly when theme changes.  
Update semantic tokens and let aliases propagate.
```

```
## Validation
```

```
Run:
```

- alias cycle detection
- missing mode detection
- hardcoded value detection
- orphan token detection

这个 Skill 把“整体主题变了，组件哪里跟着变”做成系统能力，而不是 prompt 能力。

8. Skill 5: `adk-composition-grammar`

定位：组件之间怎么组合。

输出：

```
compositions/*.pattern.json  
ontology/page-patterns.json  
rules/composition-rules.json
```



核心内容：

<> Markdown



```
---  
name: adk-composition-grammar  
description: Builds composition grammar for how components combine into product screens:  
---  
  
# Composition Grammar  
  
## Define  
  
For each pattern:  
- pattern id  
- intent  
- required components  
- optional components  
- component order  
- slot rules  
- allowed variants by context  
- responsive behavior
```

- common anti-patterns
- repair strategies

Examples

Patterns include:

- settings page
- dashboard page
- data table with filters
- onboarding flow
- pricing page
- empty state
- destructive confirmation flow

Hard vs soft

Convert only objectively checkable constraints into hard rules.

Design taste and preference should become soft rules or critic checks.

这一层避免 AI 只会“组件级正确”，但页面结构乱。

9. Skill 6: `adk-contract-author`

定位：生成组件契约。

输出：

```
components/*.contract.json
validators/component.schema.json
```



核心内容：

<> Markdown



```
---
name: adk-contract-author
description: Authors machine-readable component contracts from source snapshots, semantic
---

# Component Contract Author

## Contract sections

Every contract must include:
- id
- version
- displayName
- sourceRefs
```

```
- semanticRole
- meaning
- variants
- props
- slots
- states
- tokenBindings
- compositionRules
- pencil mapping
- code mapping
- examples
- antiExamples
- hardRules
- softRules
- sourceEvidence
- confidence
```

```
## Contract policy
```

Do not write long prose when structured fields can represent the same knowledge.

Every hardRule must be expressible as:

```
- schema rule
- graph rule
- token rule
- slot rule
- validator function
- or explicit unresolved rule
```

```
## Output
```

Write one file per component:

```
components/{component-id}.contract.json
```

这个 Skill 是 ADK 的骨架。

10. Skill 7: `adk-contract-reviewer`

定位：审查 AI 生成的契约。

它很重要，因为生产环境不能只相信 contract-author。

输出：

```
reports/contract-review/*.md
reports/contract-review/*.json
```



核心内容：

</> Markdown



```
---
name: adk-contract-reviewer
description: Reviews AI-generated component contracts for contradictions, unsupported ass
---

# Contract Reviewer

## Review checklist

For every contract, check:
- Does semanticRole match actual usage?
- Are variants semantically distinct?
- Are props mapped to Figma, Pencil, and code?
- Are token bindings complete?
- Are forbidden overrides specified?
- Are slots strict enough?
- Are hardRules actually machine-checkable?
- Are antiExamples present?
- Are low-confidence claims marked for human review?

## Output

Classify issues as:
- blocking
- risky
- improvement
- question

Do not modify contracts directly unless explicitly asked.
```

这个 Skill 相当于设计系统 reviewer。

11. Skill 8: `adk-pencil-library-compiler`

定位：把 AI Design Kit 编译成 Pencil library。

输入：

```
components/*.contract.json
tokens/token-graph.json
mappings/kit-to-pencil.json
```



输出：

```
pencil/product.lib.pen
pencil/examples.pen
```



核心内容：

Markdown



```
---
name: adk-pencil-library-compiler
description: Compiles AI Design Kit component contracts and token graphs into Pencil .lib
---

# Pencil Library Compiler

## Compilation target

Generate or update:
- pencil/product.lib.pen
- Pencil variables
- Pencil themes
- reusable components
- slot frames
- allowed descendant override paths
- metadata for component identity

## Rules

- Registered components must become reusable components.
- Instances must use type: "ref".
- Editable content must use descendants override.
- Tokenized values must use variables.
- Slot areas must preserve allowed component metadata.
- Do not compile raw imitation components.

## Metadata

Every reusable component root must include:
- componentId
- contractVersion
- figmaKey
- codeComponent
- allowedOverrides
- tokenBindings
- slotContracts
```

这就是 Figma/组件库到 Pencil 的真正桥梁。

12. Skill 9: adk-pencil-page-designer

定位：用 ADK 设计实际页面，但只输出语义操作，不直接乱画。

输出：

```
semantic/generated/*.graph.json
semantic/generated/*.ops.json
pencil/generated/*.pen
reports/design-rationale/*.md
```



核心内容：

Markdown



```
---
name: adk-pencil-page-designer
description: Generates production UI screens using the AI Design Kit and Pencil. It creat
---

# Pencil Page Designer

## Mandatory process

1. Interpret user intent.
2. Retrieve relevant ontology, patterns, components, and token subgraph.
3. Produce semantic plan.
4. Produce semantic component operations.
5. Validate operations before compiling.
6. Compile to Pencil using refs and descendants.
7. Produce design rationale.

## Forbidden

- Do not create raw visual components.
- Do not directly use Pencil batch_design for component-like UI.
- Do not hardcode tokenized values.
- Do not detach component instances.
- Do not override forbidden descendant paths.

## Allowed operation types

- component.insert
- component.setProp
- component.setVariant
- component.fillSlot
- component.reorder
- component.bindData
- token.reference
- theme.switch
- layout.applyPattern
```

这个 Skill 是实际“AI 做设计”的入口。

13. Skill 10: adk-pencil-validator

定位：把 Pencil 输出变成可审计结果。

输入：

```
pencil/generated/*.pen
components/*.contract.json
tokens/token-graph.json
```



输出：

```
reports/validation/*.json
reports/visual-review/*.md
```



核心内容：

</> Markdown



```
---
name: adk-pencil-validator
description: Validates Pencil .pen outputs against the AI Design Kit: component identity,
---

# Pencil Validator

## Static validation

Check:
- all system UI uses registered refs
- no raw imitation components
- no hardcoded colors if token exists
- descendant overrides are allowed
- slot children are allowed
- variants exist
- props satisfy contract
- token variables exist
- theme modes are complete

## MCP validation

When Pencil MCP is available, run:
- batch_get
- get_variables
- snapshot_layout
- get_screenshot

## Output

Write:
```

- validation report JSON
- visual review markdown
- blocking issue list
- suggested repair ops

这个 Skill 决定“能不能交付”。

14. Skill 11: `adk-code-compiler`

定位：从同一个 **semantic graph** 生成代码，不从 **Pencil** 反推。

输入：

```
semantic/*.graph.json
components/*.contract.json
mappings/kit-to-react.json
```

输出：

```
generated-code/*
reports/code-validation/*.json
```

核心内容：

`</>` Markdown

```
---
name: adk-code-compiler
description: Compiles semantic design graphs into production code using real component AF
---

# Code Compiler

## Source of truth

Use semantic graph and component contracts.
Do not use Pencil .pen as the primary source for code.

## Compile

Map:
- component id -> code import
- variant -> code prop
- props -> code props
- slots -> children composition
- tokens -> CSS variables / theme references
- states -> component props or state bindings
```

```
## Validate
```

```
Check:
```

- no one-off CSS imitation of system components
- all registered components use real imports
- props satisfy component API
- tokens use approved variables
- no raw hex unless explicitly allowed

这一条非常关键：避免 Pencil → HTML 丢语义。

15. Skill 12: adk-feedback-learner

定位：把人类纠错固化成系统知识。

输入：

```
用户反馈
设计 review comment
验证失败报告
人工修改 diff
```



输出：

```
components/*.contract.json patch
rules/*.json patch
examples/bad/*.json
examples/good/*.json
memory/corrections.jsonl
```



核心内容：

```
</> Markdown
```



```
---
name: adk-feedback-learner
description: Converts human design feedback and validation failures into durable AI Design
---

# Feedback Learner

## Policy

Never store important corrections only in conversation memory.

Every correction must become one or more of:
```

```
- component contract update
- composition rule
- token rule
- good example
- bad example
- repair strategy
- validator test
```

```
## Classification
```

```
Classify feedback as:
```

```
- semantic correction
- usage correction
- token correction
- visual correction
- composition correction
- code mapping correction
- Pencil mapping correction
```

```
## Output
```

```
Produce:
```

```
- patch summary
- files changed
- affected components
- validators to rerun
```

这个 Skill 让系统越用越准。

16. Skill 13: `adk-release-manager`

定位：管理 AI Design Kit 版本。

输出：

```
kit.json
reports/release/*.md
```

核心内容：

```
</> Markdown
```

```
---
name: adk-release-manager
description: Versions and releases AI Design Kit artifacts. Use when publishing a new kit
---

# Release Manager
```

```
## Release package

A release must include:
- kit.json
- contracts
- token graph
- composition rules
- Pencil library
- mappings
- validators
- examples
- changelog

## Versioning

Classify changes:
- patch: docs/examples/corrections
- minor: new component or non-breaking rule
- major: renamed component, removed variant, changed slot contract, changed token semanti

## Checks

Before release:
- validate all contracts
- validate token graph
- compile Pencil library
- run fixture pages
- compare screenshots where available
```

这个 Skill 保证生产可维护。

17. 这组 Skills 怎么协作

场景 A：首次接入一个已有产品组件库

用户命令：

```
$adk-orchestrator Build an AI Design Kit from our existing Figma library, Code Contr
```



执行链路：

```
adk-source-ingestion
  → sources/*.snapshot.json

adk-component-semantics
  → ontology/*.json
```



```
adk-token-graph
  → tokens/*.json

adk-composition-grammar
  → compositions/*.json

adk-contract-author
  → components/*.contract.json

adk-contract-reviewer
  → reports/contract-review

adk-pencil-library-compiler
  → pencil/product.lib.pen

adk-release-manager
  → kit.json
```

场景 B：生成一个真实页面

用户命令：

```
$adk-orchestrator Design a user permissions management page using our product compo
```



执行链路：

```
adk-pencil-page-designer
  → semantic plan
  → semantic ops
  → pencil/generated/user-permissions-page.pen

adk-pencil-validator
  → validation report
  → screenshot / layout report

adk-code-compiler
  → React page using actual components
```



场景 C：设计师纠错

用户命令：

```
$adk-feedback-learner In permission pages, destructive actions must be inside a cor
```



执行链路：

```
adk-feedback-learner
  → update ontology/rules/contracts/examples
```



```
adk-contract-reviewer
  → check conflicts

adk-release-manager
  → new kit version
```

18. MCP 配置应该怎么配

`.codex/config.toml` 大概会需要这些 MCP：

❏ TOML

```
[mcp_servers.figma]
url = "https://mcp.figma.com/mcp"
bearer_token_env_var = "FIGMA_OAUTH_TOKEN"

[mcp_servers.pencil]
command = "pencil"
args = ["mcp"]

[mcp_servers.playwright]
command = "npx"
args = ["-y", "@playwright/mcp"]

[mcp_servers.github]
command = "npx"
args = ["-y", "@modelcontextprotocol/server-github"]
env_vars = ["GITHUB_TOKEN"]
```

Codex 官方文档里的 MCP 配置支持 streamable HTTP，也支持本地 STDIO server；Figma MCP 的远程 URL 形式也在官方示例里出现过。

 OpenAI 开发者

19. 为什么这套 Skill 组合适合你的目标

它刚好解决你担心的三个问题。

第一：上下文不会爆

Codex Skills 会先只暴露 name、description、path，需要时才加载完整 `SKILL.md`；官方文档还说初始 skills list 有上下文预算，skill 被选择后才读完整说明。

 OpenAI 开发者

所以我们把系统拆成 12 个小 Skill，而不是一个巨型 prompt，是正确方向。

第二：生产环境可控

控制点不是 prompt，而是：

```
contracts
token graph
composition grammar
semantic ops
validators
Pencil compiler
code compiler
feedback learner
```



Skill 负责让 Codex 按流程调用它们。

第三：能随着真实组件库进化

设计师每次纠错，不再只是“下次记得”。它会进入：

```
contract
rule
example
anti-example
validator
release version
```



这才是 AI Design Kit 真正变聪明的方式。

20. 我建议第一版先实际创建这 5 个核心 Skills

虽然我上面列了 13 个，但第一版不用全写满。真正最先要落地的是：

```
1. adk-orchestrator
2. adk-source-ingestion
3. adk-contract-author
4. adk-pencil-library-compiler
5. adk-pencil-page-designer
6. adk-pencil-validator
7. adk-feedback-learner
```



其中最核心的是：

```
adk-contract-author
adk-pencil-page-designer
adk-pencil-validator
```



因为这三个决定了：

```
组件库怎么变成 AI Design Kit
AI 怎么基于 ADK 设计
```



21. 下一步最实用的交付物

我建议下一步不是继续讨论架构，而是直接生成一个 repo skeleton：

```
.agents/skills/adk-orchestrator/SKILL.md
.agents/skills/adk-source-ingestion/SKILL.md
.agents/skills/adk-contract-author/SKILL.md
.agents/skills/adk-pencil-library-compiler/SKILL.md
.agents/skills/adk-pencil-page-designer/SKILL.md
.agents/skills/adk-pencil-validator/SKILL.md
.agents/skills/adk-feedback-learner/SKILL.md

AGENTS.md

.codex/config.example.toml
ai-design-kit/schemas/component-contract.schema.json
ai-design-kit/schemas/semantic-ops.schema.json
ai-design-kit/schemas/token-graph.schema.json
```



这一套放进 Codex repo 后，就可以开始用真实组件库跑第一轮生产链路了。